



Reporte Técnico de Actividades Práctico-Experimentales Nro. 003

1. Datos de Identificación del Estudiante y la Práctica

Nombre del estudiante(s)	• Jose Camilo Merino Morocho • Edgar Fabricio Rios Cuenca • Ashley Patricia Mocha Valarezo • Abraham Daniel Maza Lapo
Asignatura	Matemáticas Discretas
Ciclo	1 A
Unidad	3
Resultado de aprendizaje de la unidad	Entiende los principios del análisis combinatorio que resuelven problemas reales de su cotidianidad, bajo los principios de solidaridad, transparencia, responsabilidad y honestidad.
Práctica Nro.	003
Tipo	Grupal
Título de la Práctica	Teoría de Grafos, Algoritmos de Búsqueda, Árboles y Aplicaciones Computacionales
Nombre del Docente	Mario Enrique Cueva Hurtado
Fecha	29 de junio al 24 de julio de 2026
Horario	Lunes 07:30 – 09:30
Lugar	Aula de la Carrera de Computación / Aula Virtual (Zoom)
Tiempo planificado en el Sílabo	12 horas (2 horas por sesión APE)

2. Objetivo(s) de la Práctica

- Comprender los fundamentos de la teoría de grafos: definición, terminología (vértices, aristas, grado, adyacencia, incidencia), tipos de grafos (dirigidos, no dirigidos, ponderados, completos, bipartitos) y representaciones (matriz de adyacencia, lista de adyacencia).
- Aplicar algoritmos fundamentales de búsqueda en grafos: BFS (Búsqueda a lo Ancho), DFS (Búsqueda en Profundidad) y Dijkstra (camino más corto), comprendiendo su funcionamiento paso a paso.

- Entender la teoría de árboles: definición, propiedades, tipos de árboles (binarios, BST, AVL), recorridos (inorden, preorden, posorden) y aplicaciones en ciencias de la computación.
- Utilizar software de modelado de grafos (Graph Online, Gephi) para representar, visualizar y analizar redes y estructuras de datos.
- Integrar los conocimientos de las tres unidades mediante un proyecto que aplique la lógica matemática, el álgebra de Boole y la teoría de grafos a un problema computacional real.

3. Materiales, Reactivos, Pizarra y marcadores de colores

- Proyector multimedia
- Hojas de trabajo con ejercicios de teoría de grafos y árboles
- Guía paso a paso de los algoritmos BFS, DFS y Dijkstra
- Plantillas para construcción de grafos y árboles
- Investigación sobre aplicaciones de grafos en redes y optimización

4. Equipos y herramientas

- PC o laptop (mínimo 1 por estudiante)
- Plataforma EVA de la UNL
- Laboratorio de computación
- Software Graph Online (<https://graphonline.ru/es>) o Gephi para modelado de grafos
- Herramientas colaborativas: Google Drive, Wiki
- Software de presentación (PowerPoint, Canva)
- Máximo de personas por grupo: 4-5 estudiantes

4. Procedimiento / Metodología Ejecutada

FASE 1: Fundamentos de la Teoría de Grafos (Semana 13: 29 junio - 03 julio 2026)

1. Los estudiantes, en equipos, construyen grafos a partir de especificaciones del lenguaje natural (ej: modelar la red de amigos de un grupo social, la red de transporte de una ciudad, la topología de una red de computadoras), identificando vértices, aristas, grados y propiedades estructurales.
2. Modelado de una red sencilla mediante software Graph Online o Gephi: los equipos crean (Usar uno creado) un grafo representando contagios de una pandemia en un grupo de personas, visualizándolo con el software y analizando sus propiedades (conectividad, caminos mínimos, nodos críticos).



FASE 2: Algoritmos de Búsqueda y Caminos Mínimos (Semana 14: 06-10 julio 2026)

1. Taller colaborativo de aplicación de algoritmos: los equipos ejecutan manualmente los algoritmos BFS, DFS y Dijkstra sobre grafos de ejemplo (no dirigidos para BFS/DFS, ponderados para Dijkstra), documentando cada paso: cola/pila de nodos visitados, orden de descubrimiento, distancias acumuladas y caminos resultantes.
2. Los equipos modelan redes mediante software (Graph Online, Gephi u otros), ingresando grafos ponderados y ejecutando el algoritmo de Dijkstra para encontrar el camino más corto entre pares de nodos. Comparan los resultados manuales con los obtenidos por el software.

FASE 3: Árboles y Estructuras de Datos Arbóreas (Semana 15: 13-17 julio 2026)

1. Los estudiantes construyen árboles binarios de búsqueda (BST) paso a paso, insertando valores numéricos y realizando los tres tipos de recorridos, documentando el orden de visita de nodos en cada caso.
2. Los estudiantes investigan de forma autónoma sobre la relación entre la teoría de grafos/árboles y las estructuras de datos utilizadas en programación (heaps, tries, árboles B), bases de datos (índices) e inteligencia artificial (árboles de decisión). Elaboran un mini-informe con exposición apoyada en recursos TIC.

FASE 4: Proyecto Integrador - Modelado y Optimización de Redes (Semana 16: 20-24 julio 2026)

1. Los equipos elaboran un informe técnico del proyecto que incluya: descripción del problema, modelado matemático (lógica, booleana, grafos), implementación en software, resultados obtenidos, análisis de optimización y conclusiones. (ARTICULO)

FASE 5: Exposición, Defensa y Evaluación Final (Semana 17: 27-31 julio 2026)

ATENCIÓN: Esta semana está destinada a actividades de fin de ciclo (29-31 julio: socialización de proyectos de investigación, casas abiertas, ferias científicas). Las actividades APE se realizan al inicio de la semana (27-28 julio).

1. Exposición y defensa oral del proyecto integrador ante la clase: cada equipo presenta su trabajo (10-15 minutos), explicando el problema abordado, la metodología aplicada (lógica + booleana + grafos), los resultados obtenidos y las conclusiones.
2. Reflexión final: los estudiantes elaboran una autoevaluación individual sobre su proceso de aprendizaje durante las tres unidades, identificando fortalezas, debilidades y metas de mejora para el siguiente ciclo académico(individual).

5. Resultados

FASE 1: Fundamentos de la Teoría de Grafos (Semana 13: 29 junio - 03 julio 2026)

1. Los estudiantes, en equipos, construyen grafos a partir de especificaciones del lenguaje natural (ej: modelar la red de amigos de un grupo social, la red de transporte de una ciudad, la topología de una red de computadoras), identificando vértices, aristas, grados y propiedades estructurales.

CONTAGIOS DE UNA PANDEMIA EN UN GRUPO DE PERSONAS

Durante una pandemia para poder comprender mejor cómo se propaga una enfermedad, se utilizará un grafo, donde cada persona será representada por un vértice y cada contacto entre dos personas será representado por una arista. De esta forma, se podrá analizar la red de contagios e identificar las personas con mayor riesgo de transmitir la enfermedad. Supongamos que en un grupo de **7 personas** una de ellas estaba infectada y, después de varios días, se registraron los siguientes contactos:

- Ana tuvo contacto con Bruno y Carlos.
- Bruno tuvo contacto con Ana, Diana y Elena.
- Carlos tuvo contacto con Ana y Fernando.
- Diana tuvo contacto con Bruno y Gabriela.
- Elena tuvo contacto con Bruno y Gabriela.
- Fernando tuvo contacto con Carlos y Gabriela.
- Gabriela tuvo contacto con Diana, Elena y Fernando.

CANTIDAD DE VERTICES	
Vértice	Personas
A	Ana
B	Bruno
C	Carlos
D	Diana
E	Elena
F	Fernando
G	Gabriela

Hay 7 vértices porque existen 7 personas:



$V = \{A, B, C, D, E, F, G\}$

CANTIDAD DE ARISTAS	
Arista	Significado
A – B	Ana tuvo contacto con Bruno
A – C	Ana tuvo contacto con Carlos
B – D	Bruno tuvo contacto con Diana
B – E	Bruno tuvo contacto con Elena
C – F	Carlos tuvo contacto con Fernando
D – G	Diana tuvo contacto con Gabriela
E – G	Elena tuvo contacto con Gabriela
F – G	Fernando tuvo contacto con Gabriela

Hay 8 aristas, ya que existen ocho contactos entre las personas.

$E = \{(A,B), (A,C), (B,D), (B,E), (C,F), (D,G), (E,G), (F,G)\}$

CANTIDAD DE GRADOS		
Persona	Contactos	Grado
Ana	Bruno, Carlos	2
Bruno	Ana, Diana, Elena	3
Carlos	Ana, Fernando	2
Diana	Bruno, Gabriela	2
Elena	Bruno, Gabriela	2
Fernando	Carlos, Gabriela	2
Gabriela	Diana, Elena, Fernando	3

El grado indica cuántos contactos tuvo cada persona.

PESO	
Arista	Peso
Ana – Bruno	5

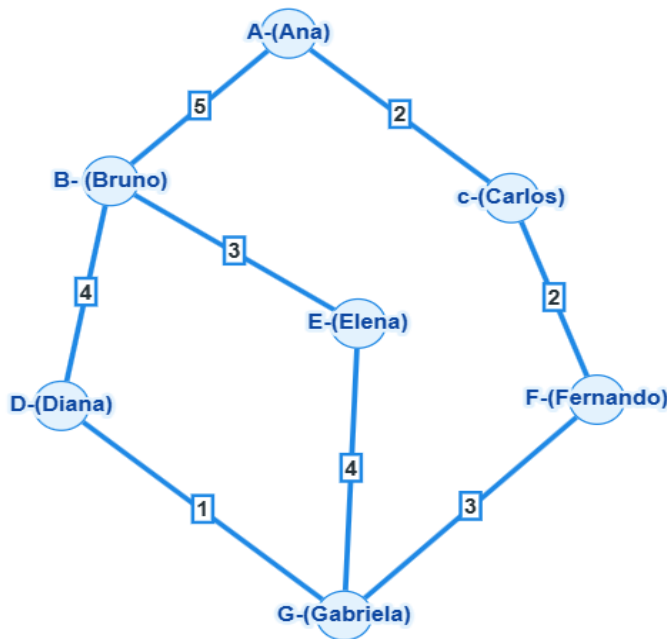


Ana – Carlos	2
Bruno – Diana	4
Bruno – Elena	3
Carlos – Fernando	2
Diana – Gabriela	1
Elena – Gabriela	4
Fernando – Gabriela	3

Aquí detallamos el numero de veces que se encuentran entre cada persona del grupo de amigos.

Propiedades estructurales

- **Tipo de grafo:** No dirigido, porque el contacto entre dos personas es mutuo.
 - **Grafo conectado:** Sí, todas las personas están relacionadas mediante uno o varios contactos.
 - **Vértices con mayor grado:** Bruno y Gabriela, ya que tuvieron **3 contactos** cada uno.
 - **Posibles focos de propagación:** Bruno y Gabriela podrían ser las personas con mayor probabilidad de transmitir la enfermedad debido a que estuvieron en contacto con más personas.
2. **Modelado de una red sencilla mediante software Graph Online o Gephi:** los equipos crean (Usar uno creado) un grafo representando contagios de una pandemia en un grupo de personas, visualizándolo con el software y analizando sus propiedades (conectividad, caminos, mínimos, nodos críticos)



Análisis de Propiedades:

Propiedad	Descripción	Resultado en el grafo
Conectividad	Indica si todos los vértices están conectados mediante uno o varios caminos.	El grafo es conectado , ya que desde cualquier persona es posible llegar a cualquier otra siguiendo las aristas. No existen vértices aislados ni componentes separadas.
Camino mínimo	Es la ruta con el menor costo total, calculado como la suma de los pesos de las aristas.	De Ana (A) a Gabriela (G) existen tres rutas: $A \rightarrow B \rightarrow D \rightarrow G = 10$, $A \rightarrow B \rightarrow E \rightarrow G = 12$ y $A \rightarrow C \rightarrow F \rightarrow G = 7$. El camino mínimo es $A \rightarrow C \rightarrow F \rightarrow G$, con un costo total de 7.
Nodos críticos	Son los vértices cuya eliminación afecta la comunicación o la propagación dentro de la red.	Bruno (B) y Gabriela (G) son los nodos críticos, ya que poseen el mayor número de conexiones (grado 3) y enlazan diferentes partes del grafo. Si uno de ellos se elimina o se aísla, la red pierde conectividad y la propagación cambia significativamente.

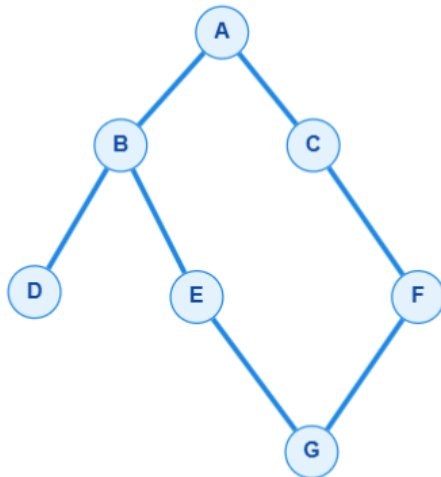
FASE 2: Algoritmos de Búsqueda y Caminos Mínimos (Semana 14: 06-10 julio 2026)

1. Taller colaborativo de aplicación de algoritmos: los equipos ejecutan manualmente los algoritmos BFS, DFS y Dijkstra sobre grafos de ejemplo (no dirigidos para BFS/DFS, ponderados para Dijkstra), documentando cada paso:

cola/pila de nodos visitados, orden de descubrimiento, distancias acumuladas y caminos resultantes.

Taller colaborativo de aplicación de algoritmos

Se utilizó el siguiente grafo para aplicar manualmente los algoritmos BFS, DFS y Dijkstra.



Algoritmo BFS (Breadth First Search)

El algoritmo BFS o búsqueda en anchura recorre un grafo visitando primero todos los nodos cercanos al nodo inicial antes de continuar con los siguientes niveles. Para ello utiliza una cola (FIFO).

- **Inicio:** Se comienza en el nodo origen **A**, el cual se marca como visitado/descubierto y se ingresa en la cola. Inmediatamente se extrae y se agregan sus vecinos contiguos directos **B** y **C** a la cola.
- **Paso 1:** Se procesa el siguiente elemento al frente de la cola, el nodo **B**, marcándolo como visitado. Se inspeccionan sus vecinos no descubiertos y se añaden al final de la cola los nodos **D** y **E**.
- **Paso 2:** Se extrae de la cola el nodo **C**. Se explora su adyacencia y se incorpora el nodo **F** a la estructura.
- **Paso 3:** Se continúa el vaciado secuencial de la cola procesando consecutivamente los nodos **D** y **E**. Al evaluar los vecinos de **E**, se descubre el nodo **G** y se encola.
- **Paso 4:** Finalmente, se extraen y procesan los nodos restantes **G** y **F**. Como todos los nodos del grafo han sido completamente descubiertos y validados, la cola queda vacía y el algoritmo concluye.

Resultado: Orden de descubrimiento

$A \rightarrow B \rightarrow C \rightarrow D \rightarrow E \rightarrow G \rightarrow F$

BFS permitió recorrer el grafo por niveles, visitando primero todos los vecinos del nodo inicial antes de continuar con los demás.

Algoritmo DFS (Depth First Search)

El algoritmo DFS (Depth First Search) o búsqueda en profundidad explora un camino hasta donde sea posible antes de regresar para continuar con otra rama. Utiliza una pila (LIFO).

- **Inicio:** Se define el punto de partida en el nodo **A**, colocándolo en el tope de la pila de exploración.
- **Paso 1:** Desde el nodo **A** se avanza profundamente hacia su primer vecino disponible **B**. Siguiendo la política de máxima profundidad, desde **B** se progresa de inmediato hacia **D**, alcanzando un nodo hoja (terminal) dentro de esta ramificación.
- **Paso 2:** Al no existir más vecinos sin visitar desde **D**, se ejecuta el mecanismo de retroceso o *backtracking* regresando la ejecución hasta el nodo anterior activo (**B**). Desde ahí se redirige la exploración hacia la siguiente ruta inexplorada: el nodo **E**.
- **Paso 3:** Desde el vértice **E** se continúa el avance lineal hacia el nodo **G**. De forma sucesiva, desde **G** se explora hacia **F**, y finalmente desde **F** se alcanza el nodo **C**, completando de forma exhaustiva la totalidad de los componentes del grafo.

Resultado: Orden de descubrimiento

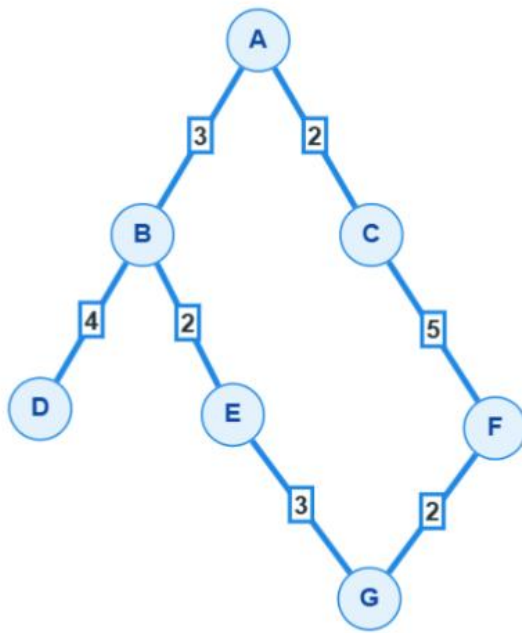
$A \rightarrow B \rightarrow D \rightarrow E \rightarrow G \rightarrow F \rightarrow C$

DFS exploró completamente cada rama antes de regresar, permitiendo recorrer el grafo en profundidad.

Algoritmo de Dijkstra (Camino más corto)

El algoritmo de Dijkstra permite encontrar el camino de menor costo entre un nodo origen y un nodo destino en un grafo ponderado. Para ello, selecciona en cada iteración el nodo con la menor distancia acumulada y actualiza las distancias de sus nodos vecinos hasta obtener la ruta más corta.

Grafo ponderado



Arista	Peso
A – B	3
A – C	2
B – D	4
B – E	2
C – F	5
E – G	3
F – G	2

Nodo origen: A

Nodo destino: G

Desarrollo

- **Inicial:** Se inicializan las etiquetas de distancia. La distancia acumulada del nodo origen **A** es 0, mientras que la distancia provisional de todos los demás nodos restantes se establece en infinito (∞). El conjunto de nodos evaluados y definitivos se encuentra vacío.
- **Iteración 1:** Se selecciona el nodo con la distancia mínima actual (**A**, costo 0) y se marca como confirmado/permanente. Se examinan sus aristas adyacentes y se realiza la relajación: la distancia hacia **B** pasa de ∞ a 3 ($0 + 3$), y la distancia hacia **C** disminuye a 2 ($0 + 2$).
- **Iteración 2:** El nodo no confirmado con la menor distancia acumulada es **C** (costo 2). Se fija como permanente. Evaluando su periferia, se calcula la distancia acumulada proyectada hacia su vecino **F**, la cual se actualiza con un valor de 7 ($2 + 5$).
- **Iteración 3:** El nodo menor restante es **B** (costo 3). Se establece como permanente. Se inspeccionan sus vecinos directos para actualizar métricas: la distancia hacia **D** pasa a ser 7 ($3 + 4$) y la distancia óptima temporal hacia el nodo **E** se reduce a 5 ($3 + 2$).
- **Iteración 4:** Se selecciona y confirma el nodo **E** (costo 5) por poseer el menor valor provisional. Desde **E**, se evalúa el camino hacia su único vecino pendiente **G**, computando y asignando una distancia acumulada de 8 ($5 + 3$).
- **Iteración 5:** Se evalúan y fijan los nodos restantes **D** (costo 7) y **F** (costo 7). Al inspeccionar las conexiones existentes (por ejemplo, de **F** hacia **G** el costo sería $7 + 2 = 9$), se comprueba que ninguna de estas rutas alternativas mejora o reduce la distancia ya calculada previamente hacia **G**.
- **Iteración 6:** Se selecciona formalmente el nodo de destino **G** (costo 8), consolidando su estado como nodo definitivo y dando por finalizadas las operaciones del algoritmo.

Resultado

Camino mínimo: $A \rightarrow B \rightarrow E \rightarrow G$

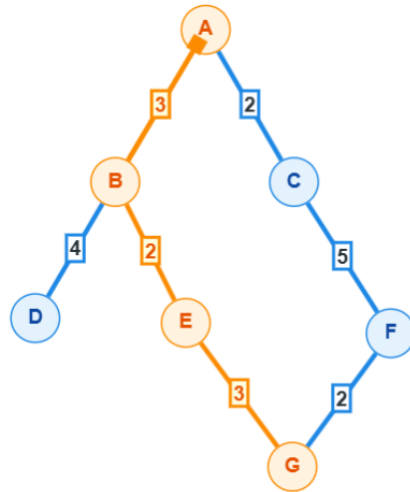
Costo total: 8

2. Los equipos modelan redes mediante software (Graph Online, Gephi u otros), ingresando grafos ponderados y ejecutando el algoritmo de Dijkstra para encontrar el camino más corto entre pares de nodos. Comparan los resultados manuales con los obtenidos por el software.

Modelado de la red mediante software

Se ingresó el mismo grafo en **Graph Online**, asignando los mismos pesos a cada arista y ejecutando el algoritmo de Dijkstra con **A** como nodo origen y **G** como nodo destino.

La longitud del camino más corto es 8: $A \rightarrow B \rightarrow E \rightarrow G$



Resultado obtenido

El software encontró el siguiente camino mínimo:

$A \rightarrow B \rightarrow E \rightarrow G$ Costo total: 8

Comparación

Característica	Manual	Software
Nodo origen	A	A
Nodo destino	G	G
Camino mínimo	$A \rightarrow B \rightarrow E \rightarrow G$	$A \rightarrow B \rightarrow E \rightarrow G$
Costo total	8	8

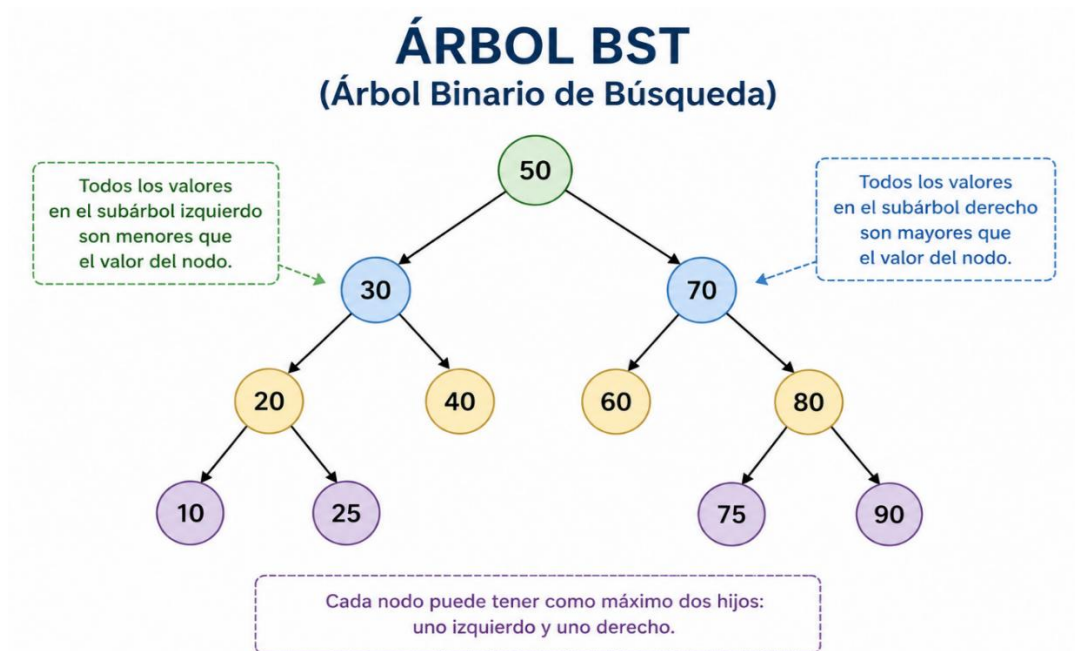
Los resultados obtenidos manualmente coinciden con los generados por **Graph Online**, comprobando que el algoritmo de Dijkstra fue aplicado correctamente. El cálculo manual permitió comprender el procedimiento paso a paso, mientras que el software facilitó la verificación del camino mínimo y del costo total.

FASE 3: Árboles y Estructuras de Datos Arbóreas (Semana 15: 13-17 julio 2026)

1. Los estudiantes construyen árboles binarios de búsqueda (BST) paso a paso, insertando valores numéricos y realizando los tres tipos de recorridos, documentando el orden de visita de nodos en cada caso.

¿Qué es un árbol binario de búsqueda (BST)?

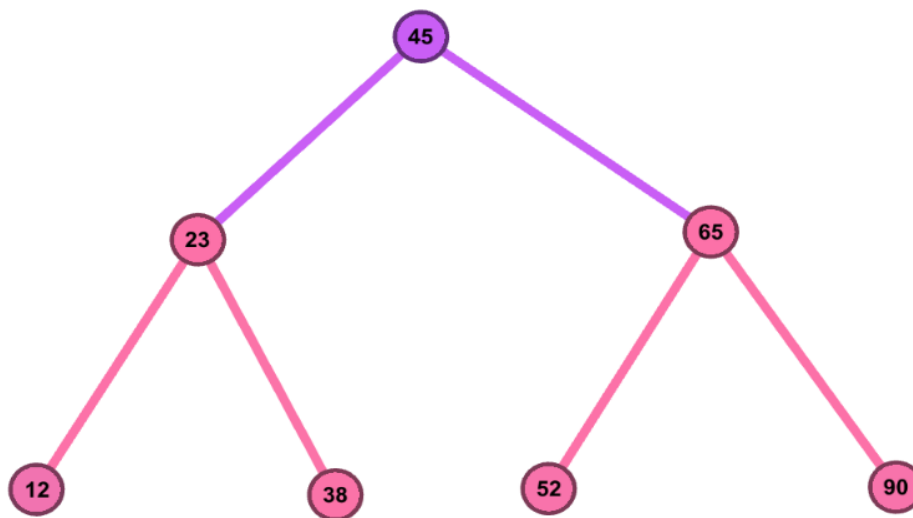
Un Árbol Binario de Búsqueda (BST) es una estructura de datos jerárquica compuesta por nodos, donde cada nodo puede tener como máximo dos hijos y se mantiene la propiedad de que los valores menores se ubican a la izquierda del nodo y los mayores a la derecha, permitiendo realizar búsquedas y operaciones de manera eficiente.



Ejercicio 1:

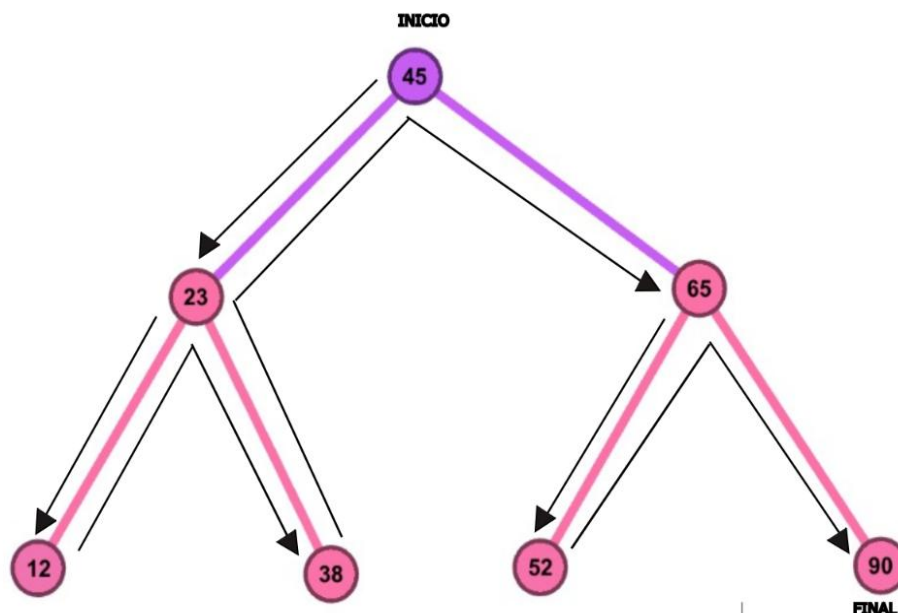
Planteamiento del problema:

Una tienda de tecnología quiere organizar los códigos de sus productos para optimizar las búsquedas en su base de datos. El sistema registra los códigos según van llegando los camiones de inventario. Se requiere construir el árbol binario de búsqueda para la siguiente secuencia de códigos de productos: [45, 23, 65, 12, 38, 52, 90] y documentar sus tres recorridos para auditoría del sistema.

**Preorden (Raíz --- Izquierda --- Derecha):**

Anotamos la raíz actual y luego bajamos.

- Anotamos la raíz principal: **45**.
- Vamos al subárbol izquierdo: anotamos **23**, bajamos a su izquierda **12**, subimos e irnos a su derecha **38**.
- Vamos al subárbol derecho: anotamos **65**, bajamos a su izquierda **52**, subimos e irnos a su derecha **90**.

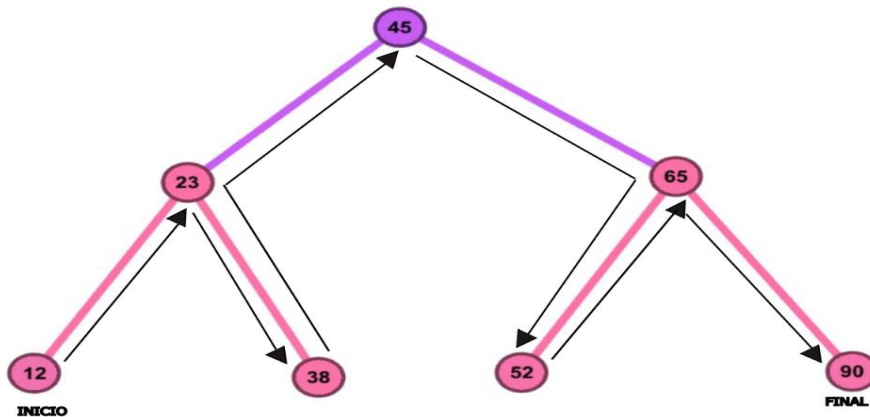


- **Resultado:** [45, 23, 12, 38, 65, 52, 90]

Inorden (Izquierda --- Raíz --- Derecha):

Avanzamos hasta el extremo izquierdo antes de anotar.

- El más a la izquierda es **12**, subimos a su padre **23**, bajamos a su derecha **38**.
- Subimos a la raíz principal **45**.
- Vamos al lado derecho: buscamos el más a la izquierda que es **52**, subimos a su padre **65**, y finalizamos en su derecha **90**.

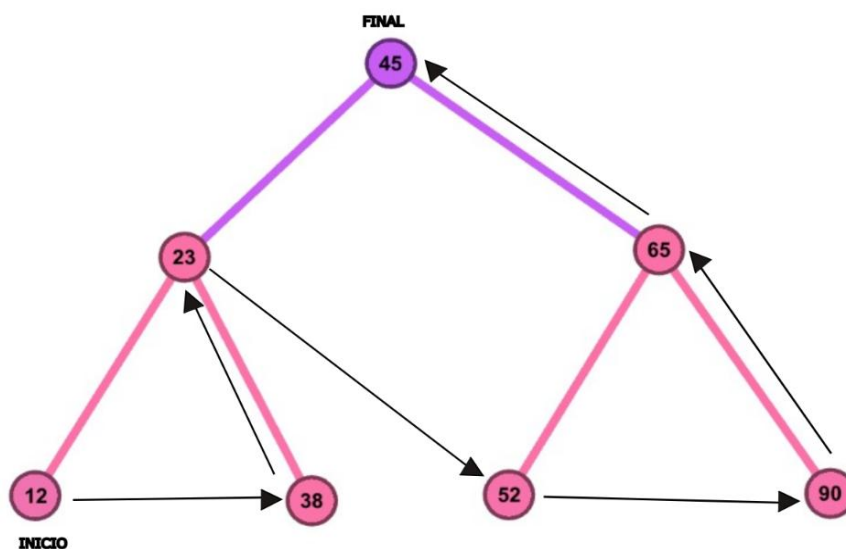


- **Resultado:** [12, 23, 38, 45, 52, 65, 90]

Postorden (Izquierda --- Derecha --- Raíz):

Los padres se procesan al final.

- Hijos del 23: anotamos **12** y **38**. Ahora anotamos al padre: **23**.
- Hijos del 65: anotamos **52** y **90**. Ahora anotamos al padre: **65**.
- Finalmente, anotamos la raíz del árbol completo: **45**.

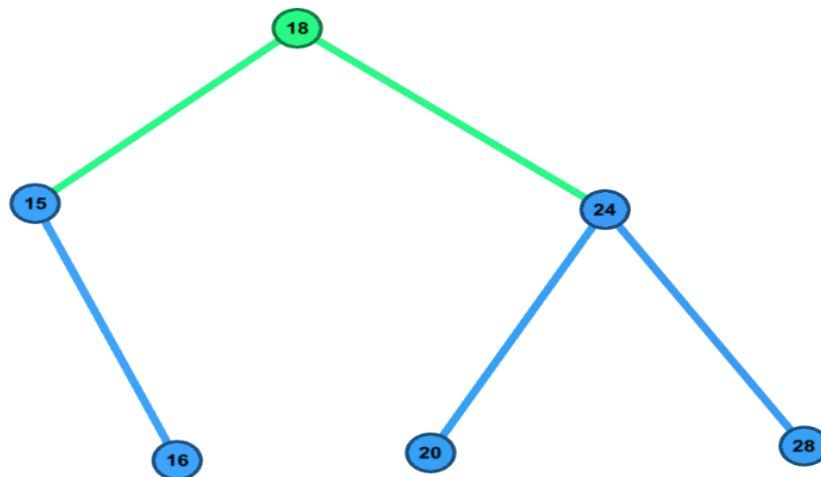


- **Resultado:** [12, 38, 23, 52, 90, 65, 45]

Ejercicio 2:

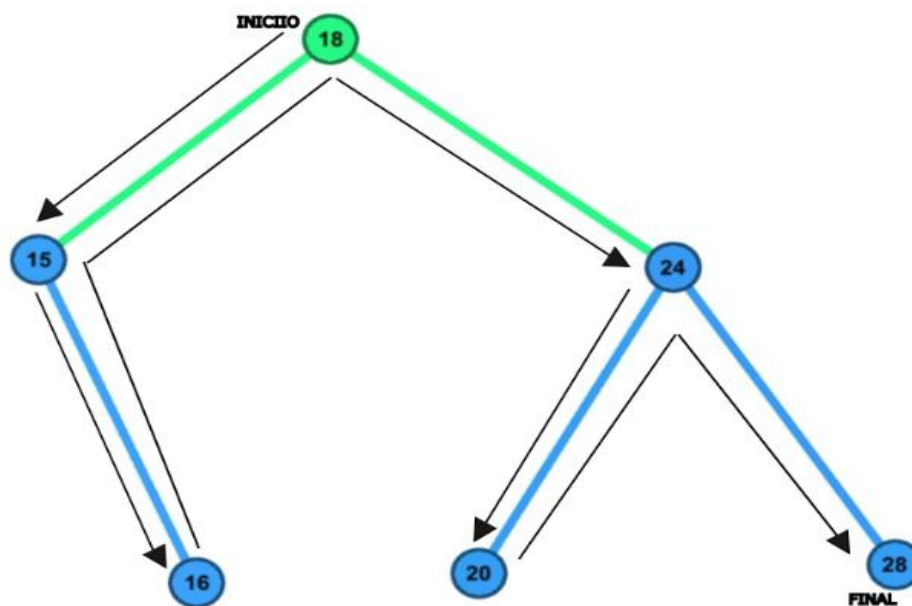
Planteamiento del problema:

Una estación meteorológica registra las temperaturas máximas del mes para evaluar las tendencias de cambio climático. Los sensores envían los datos de manera asíncrona. Se requiere construir un BST para almacenar las siguientes lecturas de temperatura en grados Celsius: [18, 24, 15, 20, 28, 16] y calcular los recorridos para que el software genere los reportes visuales.



Preorden:

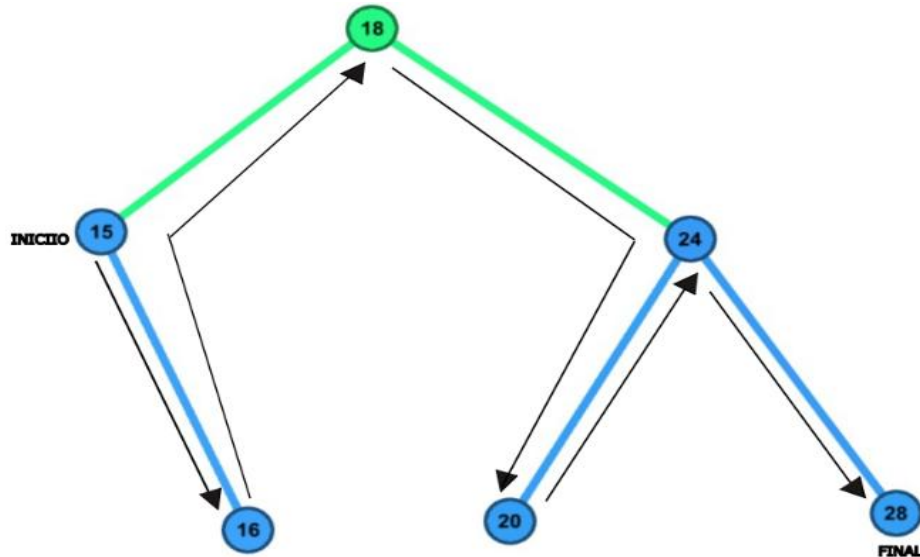
- Raíz principal: **18**.
- Subárbol izquierdo: padre **15**, luego su hijo derecho **16**.
- Subárbol derecho: padre **24**, luego su hijo izquierdo **20**, y su hijo derecho **28**.



- **Resultado:** [18, 15, 16, 24, 20, 28]

Inorden:

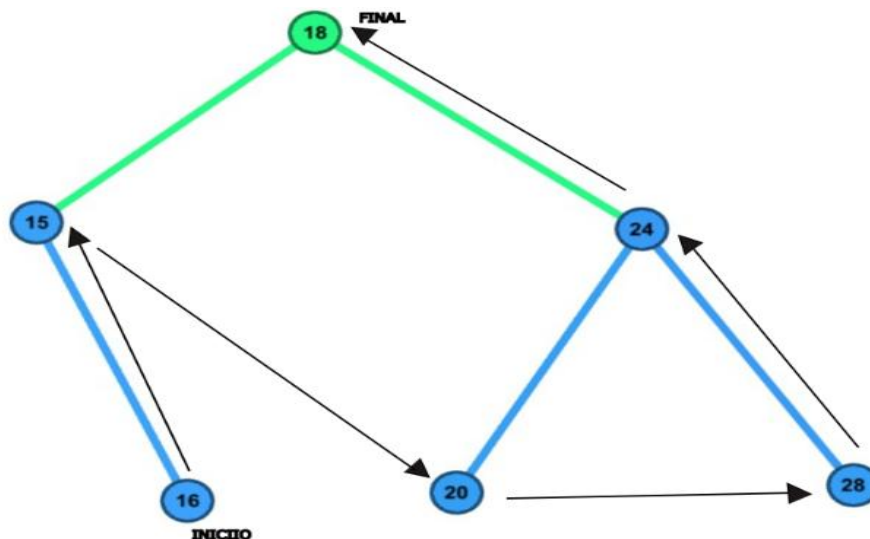
- Primero iniciamos en la raíz padre **15**, luego a su hijo derecho **16**.
- Raíz del árbol: **18**.
- Subárbol derecho: el más a la izquierda es **20**, subimos a su padre **24**, terminamos en su derecha **28**.



- **Resultado:** [15, 16, 18, 20, 24, 28]

Postorden:

- Subárbol izquierdo: el hijo **16** va antes que su padre **15**.
- Subárbol derecho: procesamos los hijos **20** y **28** antes que a su padre **24**.
- Finalmente, la raíz principal: **18**.



- **Resultado:** [16, 15, 20, 28, 24, 18]

Ejercicio 3:

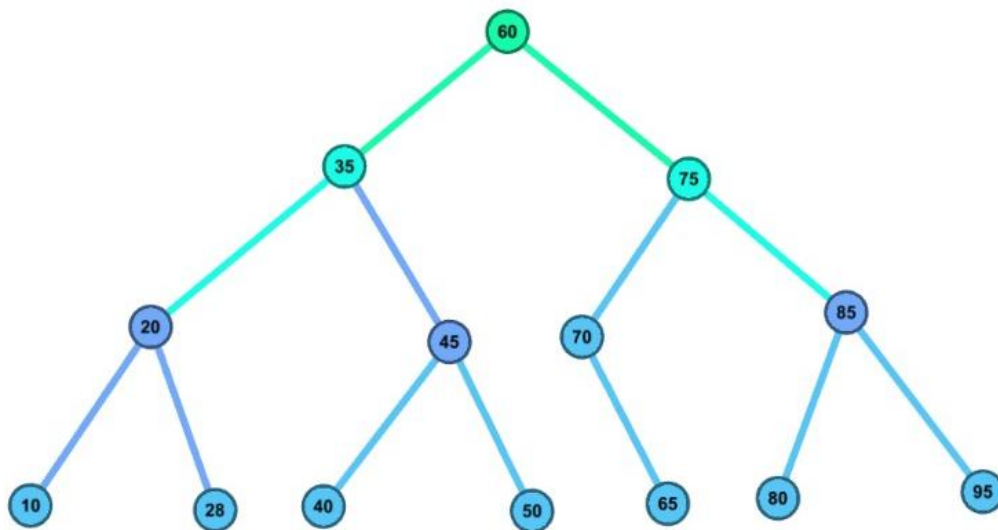
Planteamiento del Problema:

Sistema de Triage en Emergencias Médicas

Un hospital de alta complejidad implementa un sistema automatizado para priorizar la atención en la sala de emergencias. A cada paciente se le asigna un **índice de gravedad numérica** del 1 al 100 mediante un algoritmo médico en el momento en que ingresa (donde los valores representan la criticidad y compatibilidad sintomática).

Para almacenar estos índices de manera organizada y permitir consultas instantáneas, el sistema utiliza un Árbol Binario de Búsqueda (BST). Los pacientes ingresan en el siguiente orden estricto de índices:

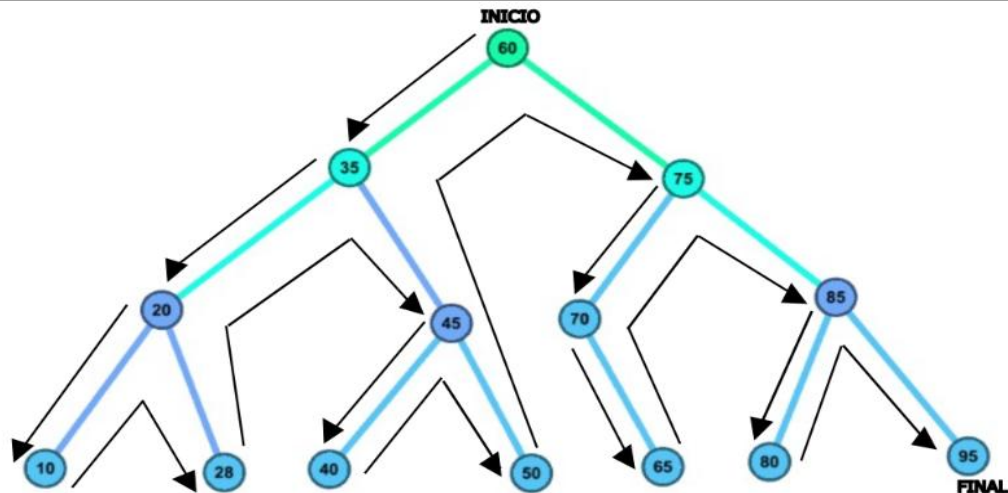
[60, 35, 75, 20, 45, 70, 85, 10, 28, 40, 50, 65, 80, 95]



PREORDEN (Raíz → Izquierda → Derecha)

Se anota el número al tocarlo por primera vez.

1. Raíz principal: **60**
2. Bloque izquierdo del 35: **35 --- 20 --- 10 --- 28**
3. Bloque derecho del 35: **45 --- 40 --- 50**
4. Bloque izquierdo del 75: **75 --- 70 --- 65**
5. Bloque derecho del 75: **85 --- 80 --- 95**

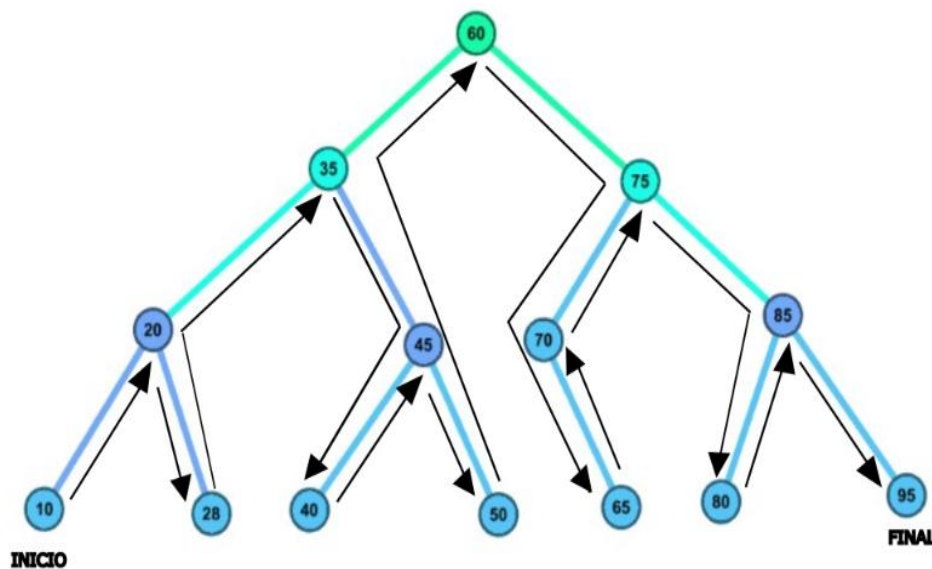


- **Resultado:** [60, 35, 20, 10, 28, 45, 40, 50, 75, 70, 65, 85, 80, 95]

INORDEN (Izquierda → Raíz → Derecha)

Se anota el número al pasar por debajo de él (de menor a mayor).

1. Extremo izquierdo (rama 20): **10 --- 20 --- 28**
2. Sube a la subraíz: **35**
3. Siguiendo rama (rama 45): **40 --- 45 --- 50**
4. Sube a la raíz principal: **60**
5. Siguiendo rama (rama 70): **65 --- 70**
6. Sube a la subraíz: **75**
7. Extremo derecho (rama 85): **80 --- 85 --- 95**

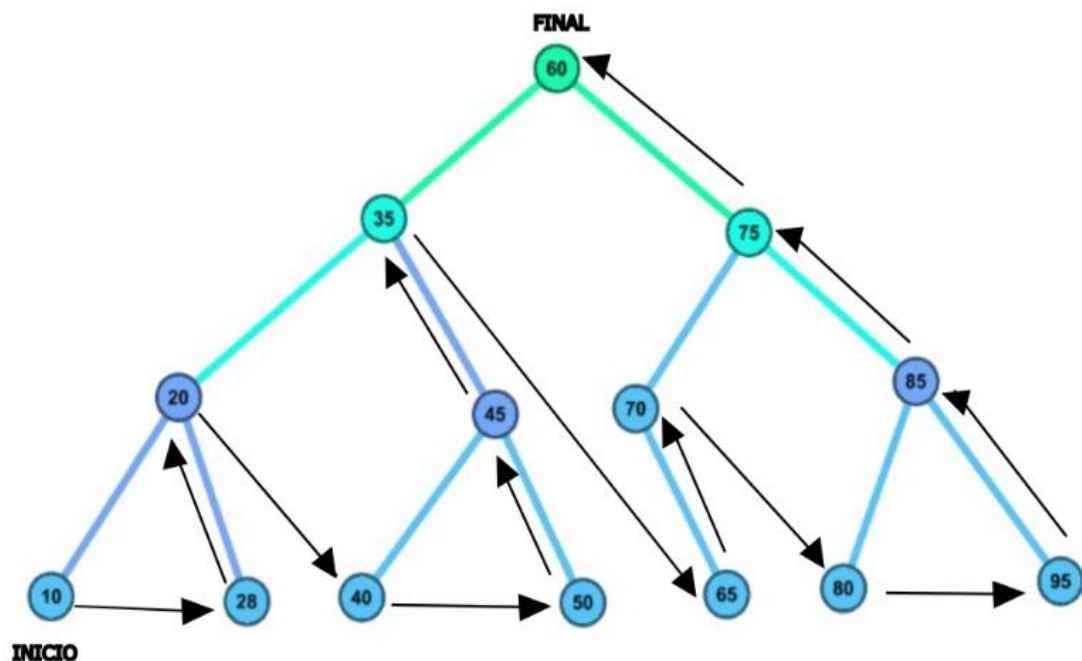


- **Resultado:** [10, 20, 28, 35, 40, 45, 50, 60, 65, 70, 75, 80, 85, 95]

POSTORDEN (Izquierda → Derecha → Raíz)

Se anotan primero los hijos y al final sus padres.

1. Hijos y padre de la rama 20: **10 --- 28 --- 20**
2. Hijos y padre de la rama 45: **40 --- 50 --- 45**
3. Sube al padre de ambos bloques: **35**
4. Hijo y padre de la rama 70: **65 --- 70**
5. Hijos y padre de la rama 85: **80 --- 95 --- 85**
6. Sube al padre de ambos bloques: **75**
7. Al final de todo, la raíz principal: **60**



- **Resultado:** [10, 28, 20, 40, 50, 45, 35, 65, 70, 80, 95, 85, 75, 60]

2. Los estudiantes investigan de forma autónoma sobre la relación entre la teoría de grafos/árboles y las estructuras de datos utilizadas en programación (heaps, tries, árboles B), bases de datos (índices) e inteligencia artificial (árboles de decisión).

Las estructuras de datos se consideran los bloques de construcción fundamentales de la informática moderna, permitiendo el almacenamiento y la manipulación eficiente de la información necesaria para diseñar algoritmos complejos. Estas estructuras no solo organizan los datos, sino que son esenciales para el rendimiento de aplicaciones en diversos dominios técnicos.

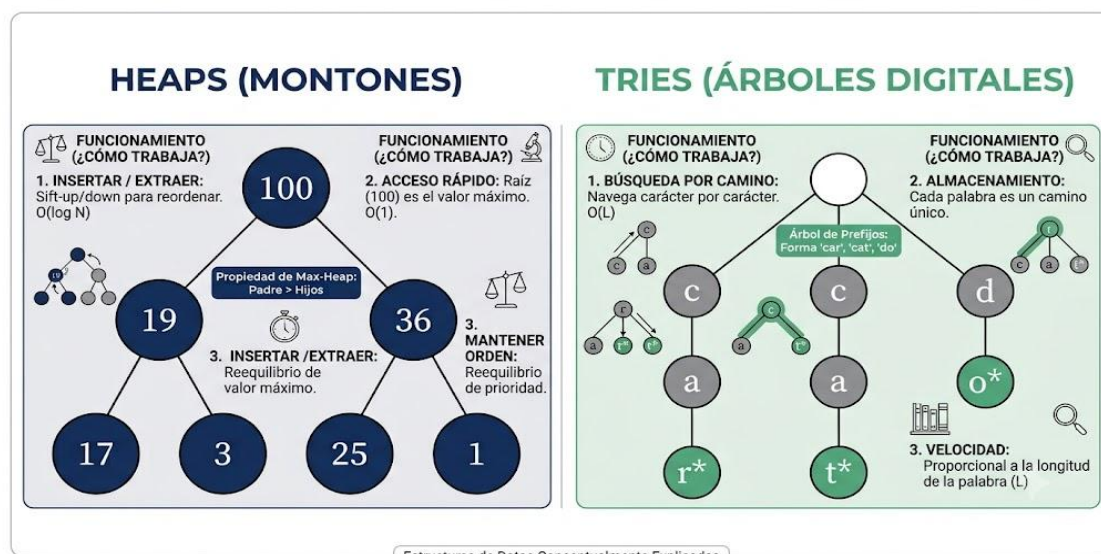
1. Fundamentos Teóricos: Árboles y Grafos

Los árboles se definen como estructuras jerárquicas en las que los nodos están conectados entre sí mediante aristas, partiendo de un único nodo raíz. Esta disposición es ideal para representar relaciones de parentesco o niveles de organización. Por su parte, los grafos consisten en vértices conectados por aristas que representan relaciones más complejas y son fundamentales para modelar redes sociales, mapas de carreteras y topologías de red.

2. Estructuras Especializadas en Programación (Heaps y Tries)

La aplicación práctica de la teoría de árboles ha dado lugar a estructuras optimizadas para tareas específicas de software:

- **Heaps (Montones):** Son árboles especializados basados en la "propiedad de montón", donde el nodo padre es consistentemente mayor o menor que sus hijos. Su importancia radica en la implementación eficiente de **colas de prioridad** y en el funcionamiento del algoritmo de **Dijkstra** para encontrar el camino más corto.
- **Tries (Árboles Digitales):** A diferencia de los árboles binarios, los tries pueden tener múltiples hijos por nodo y su altura se rige por la longitud de las claves almacenadas. Se destacan por su uso en **correctores ortográficos**, en funciones de **autocompletado** y en el enrutamiento de direcciones IP.

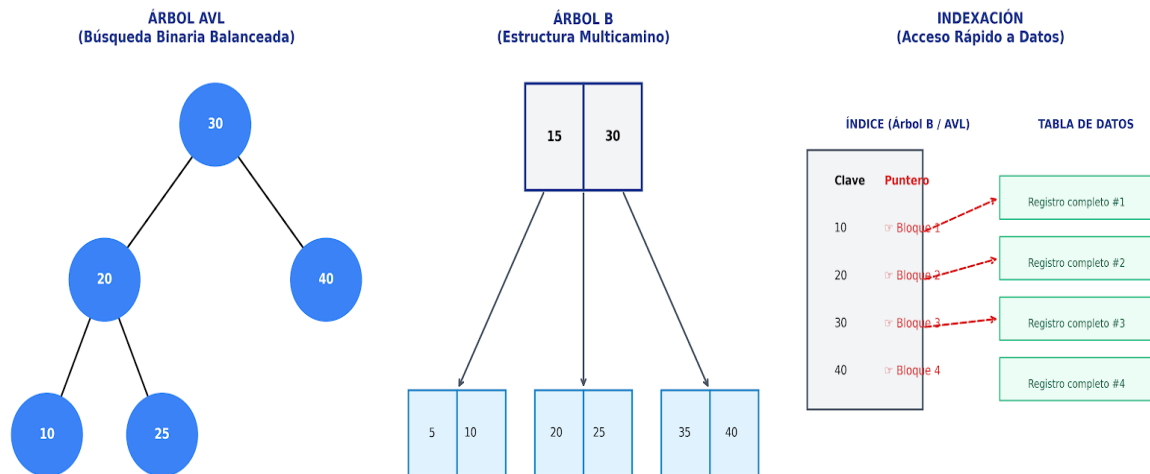


3. Gestión de Información y Bases de Datos (Árboles B e Índices)

En el ámbito del almacenamiento masivo, las estructuras deben minimizar los tiempos de acceso a memoria secundaria:

- **Árbol B:** Este tipo de árbol auto balanceado está diseñado específicamente para manejar grandes cantidades de datos en disco. El texto indica que son el estándar en **sistemas de archivos y bases de datos** donde la lectura y escritura eficientes desde el disco son críticas.

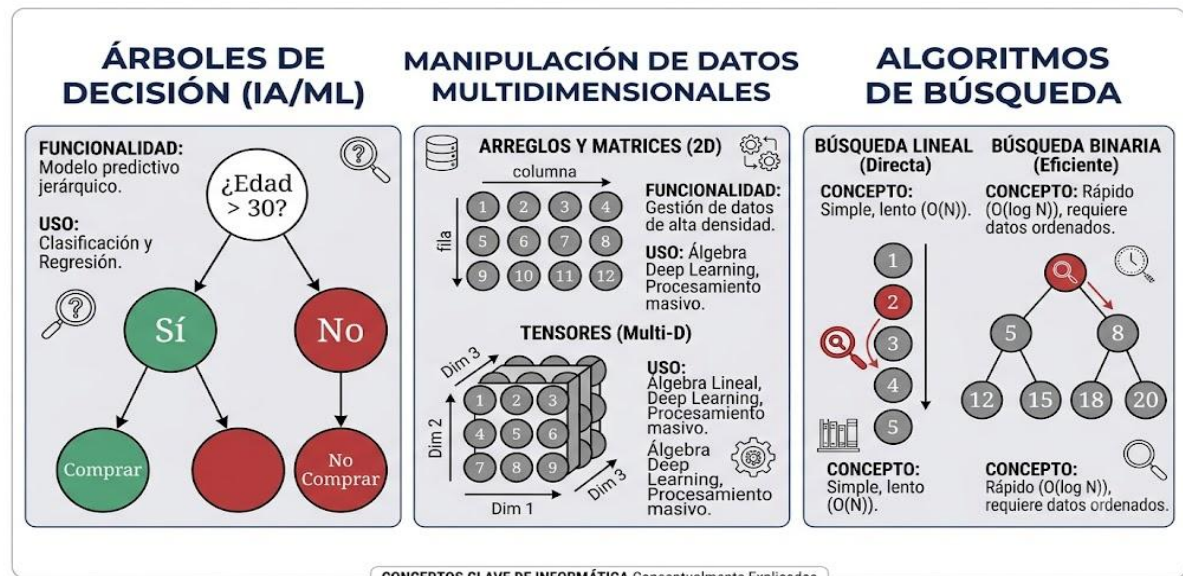
- **Árboles AVL:** Son árboles binarios de búsqueda que mantienen una diferencia mínima de altura entre sus subárboles para garantizar tiempos de búsqueda rápidos (). Se utilizan comúnmente en la **indexación de bases de datos** y en tablas de símbolos.
- **Indexación:** Se señala que las bases de datos dependen de árboles B, tablas hash y diversas formas de indexación para permitir la recuperación de registros específicos entre millones de datos de forma precisa y rápida.



4. Inteligencia Artificial y Machine Learning

El campo de la IA utiliza estas estructuras tanto para el modelado lógico como para la gestión de datos a gran escala:

- **Árboles de Decisión:** La estructura jerárquica de los árboles es la base de los algoritmos de árboles de decisión, utilizados para clasificar información y facilitar la toma de decisiones automatizada.
- **Manipulación de Datos Multidimensionales:** En el aprendizaje automático, se emplean estructuras como **arreglos, matrices y tensores** para almacenar y procesar datos masivos de múltiples dimensiones.
- **Algoritmos de Búsqueda:** La IA también se apoya en árboles binarios y tablas hash para implementar procesos eficientes de recuperación de información.



5. Optimización, Compromisos y Futuro

La elección de una estructura sobre otra implica considerar **trade-offs** o compromisos técnicos. El desarrollador debe evaluar la **complejidad temporal** (tiempo de ejecución) y la **complejidad espacial** (memoria requerida). Por ejemplo, mientras que las tablas hash ofrecen búsquedas casi instantáneas (), su mantenimiento suele ser más difícil en comparación con los arreglos simples.

Hacia el futuro, la investigación se está centrando en el desarrollo de estructuras de datos adaptadas a:

- **Big Data:** Manejo de datasets masivos en entornos distribuidos.
- **Blockchain:** Estructuras que soporten la descentralización, inmutabilidad y el consenso.
- **Computación Cuántica:** Optimización de estructuras para nuevos entornos de procesamiento.
- **Seguridad y Privacidad:** Creación de formas de almacenamiento que garanticen la protección de datos sensibles.

FASE 4: Proyecto Integrador - Modelado y Optimización de Redes (Semana 16: 20-24 julio 2026)

1. Los equipos elaboran un informe técnico del proyecto que incluya: descripción del problema, modelado matemático (lógica, booleana, grafos), implementación en software, resultados obtenidos, análisis de optimización y conclusiones. (ARTICULO)

Informe Técnico: Modelado y Optimización de Redes de Sensores Inalámbricos (WSN) mediante Grafos de Múltiples Saltos

Resumen

El presente informe técnico documenta el diseño y la optimización del consumo energético en Redes de Sensores Inalámbricos (WSN) empleando el protocolo MH-GEER (Enfoque basado en grafos de múltiples saltos para un protocolo de enrutamiento energéticamente eficiente). Al reemplazar la transmisión directa (single-hop) de LEACH por una estructura de enrutamiento basada en grafos de múltiples saltos, se mitiga de forma contundente el desgaste prematuro de los nodos lejanos a la estación base. Este trabajo integra de manera interdisciplinaria la teoría de grafos, la lógica proposicional, el álgebra de Boole y algoritmos bio-inspirados (Optimización de Colonia de Hormigas), proporcionando un análisis de estabilidad robusto respaldado por simulaciones cuantitativas y validaciones de optimización matemática clásicas basándose en la investigación de Rhim et al. (2018).

1. Descripción del Problema y Justificación Física

Las Redes de Sensores Inalámbricos (WSN) constan de micro-sensores con severas restricciones de energía, cómputo y memoria. En los protocolos de enrutamiento jerárquico tradicionales como LEACH, el nodo cabecera ("Cluster Head" o CH) de cada agrupación transmite los datos agregados directamente a la Estación Base (BS). Sin embargo, la atenuación de la señal de radiofrecuencia (fórmula de propagación en espacio libre y multi-trayecto) escala de forma no lineal con la distancia:

$$E_{Tx}(k, d) = k \cdot E_{elec} + k \cdot \epsilon_{fs} \cdot d^2 \text{ (para distancias cortas } d < d_0)$$

$$E_{Tx}(k, d) = k \cdot E_{elec} + k \cdot \epsilon_{mp} \cdot d^4 \text{ (para distancias largas } d \geq d_0)$$

Esta dependencia exponencial provoca que los CH situados en la periferia de la red agoten sus recursos de manera acelerada. La transición hacia un enfoque de múltiples saltos dinámicos es matemáticamente indispensable para fragmentar distancias críticas en trayectos intermedios más cortos y eficientes.

2. Modelado Matemático Integrador

Para dotar de formalismo e ingenio al protocolo MH-GEER, se unifican tres pilares de las matemáticas discretas (lógica, booleana, grafos) y se complementan con una validación algorítmica estricta.

2.1. Teoría de Grafos y Optimización Probabilística (ACO)

La red inter-clúster se modela como un grafo dirigido ponderado $G = (V, E)$. El conjunto de vértices V incluye a las cabeceras de clúster $\{CH_1, CH_2, \dots, CH_{N-1}\}$ y a la Estación Base (BS) como el nodo sumidero CH_N . Las aristas E definen los enlaces de comunicación disponibles.

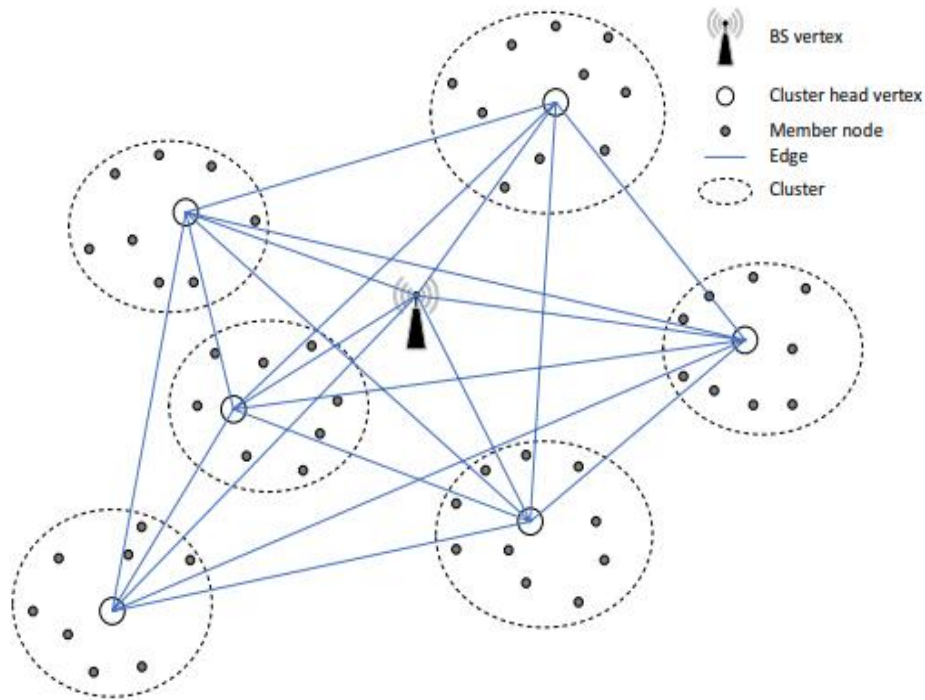


Fig. 1. Modelo gráfico de una red de cabeceras de clúster (Cluster Heads). [Rhim et al., 2018].

Los agentes de software (hormigas virtuales) seleccionan probabilísticamente el siguiente salto desde el nodo i al nodo j utilizando la regla de transición de la colonia de hormigas:

$$P_{ij} = \frac{[\tau_{ij}]^{\alpha} * [\eta_{ij}]^{\beta}}{\sum [\tau_{ij}]^{\alpha} * [\eta_{ij}]^{\beta}}$$

Donde:

- $\tau_{i,j}$: representa la concentración de feromona en la arista, correlacionada con la calidad histórica del enlace y el flujo de tráfico acumulado.
- $\eta_{i,j} = E_residual_j / d_{i,j}^2$: es la visibilidad heurística, que prioriza nodos destino con alta energía restante y distancias geométricas cortas.
- α y β : son parámetros de control que regulan la influencia relativa de la feromona frente a la heurística de proximidad.

2.2. Lógica Proposicional aplicada al Enrutamiento Libre de Bucles

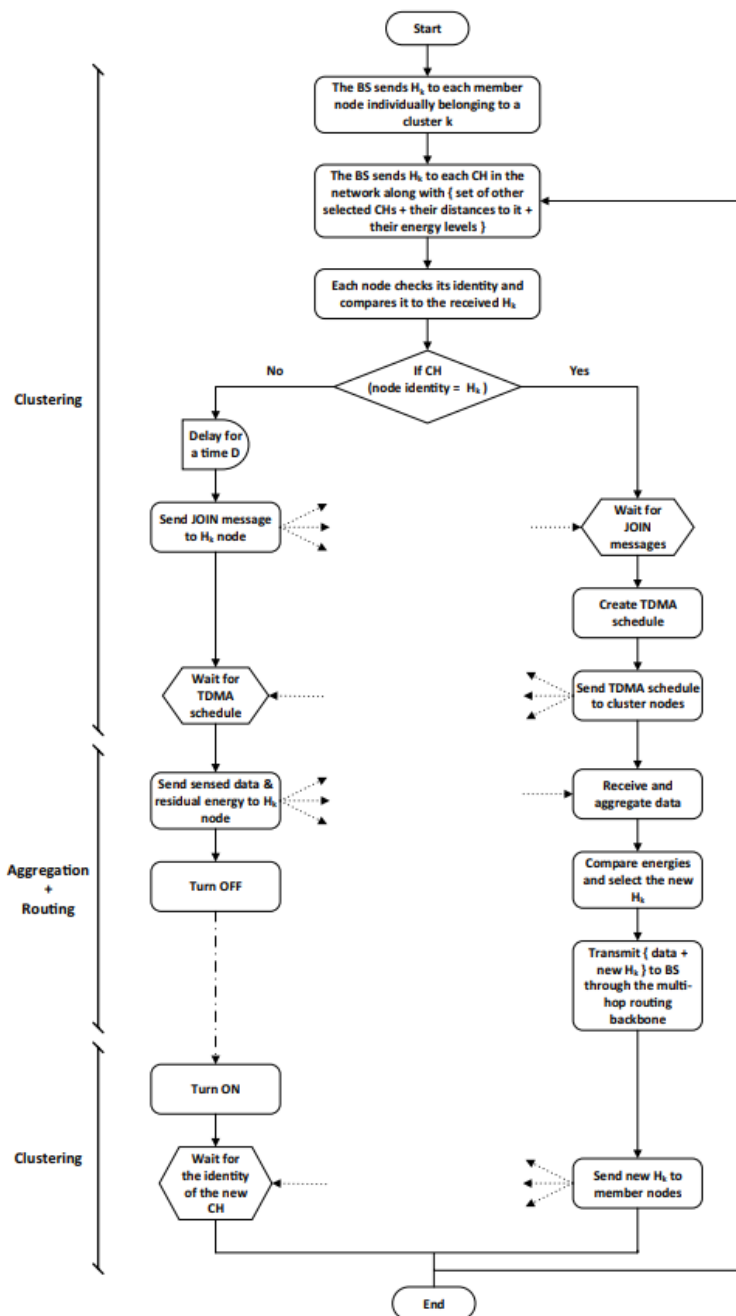


Fig. 3 Cluster heads declaration phase

Fig. 2. Diagrama de flujo de la fase de declaración de cabeceras de clúster. [Rhim et al., 2018].

Para garantizar que los paquetes de datos no queden atrapados en ciclos cerrados (loops) infinitos dentro del grafo, se evalúa de manera estricta la validez semántica del camino. Sea pth_k la secuencia ordenada de nodos visitados en la ruta actual. La proposición lógica que evalúa si un nodo candidato j puede constituir el siguiente salto es:

$$\Phi(j) = \neg(j \in \text{pth_k}) \wedge (E_{\text{residual_j}} > \theta_{\text{umbral}}) \wedge (\text{Distancia}(j, \text{BS}) < \text{Distancia}(i, \text{BS}))$$

Si $\Phi(j) \equiv \text{Verdadero}$, el enlace es lógicamente válido. La condición de parada automática del agente se define mediante la implicación lógica: $i = \text{BS} \rightarrow \text{Finalizar_Ruta}$.

2.3. Álgebra de Boole y Compuertas de Control de Rol

Cada sensor evalúa internamente una función booleana de decisión para definir su rol operativo (La lógica de decisión para la asignación de roles se detalla en el proceso algorítmico mostrado en la Fig. 2). Definimos las variables de entrada binarias:

- **A (Energía superior al promedio regional):** 1 si $E_{\text{nodo}} \geq E_{\text{promedio}}$ 0 en caso contrario.
- **B (Densidad de vecindad adecuada):** 1 si el número de nodos vecinos es óptimo, 0 si está aislado.
- **C (Frecuencia de ciclo superada):** 1 si el nodo no ha sido elegido CH recientemente.

La función de activación booleana simplificada para la designación de CH está dada por el producto lógico (conjunción estricta):

$$F(A, B, C) = A \cdot B \cdot C$$

2.4. Validación Algorítmica Paso a Paso

Antes de proceder con la implementación y codificación del protocolo distribuido, resulta indispensable validar formalmente el modelo topológico utilizando algoritmos clásicos de optimización de rutas sobre grafos. A continuación, se presenta la validación paso a paso mediante el algoritmo de Dijkstra, empleando de forma estricta los parámetros empíricos del Escenario 1 (S1) descritos en el artículo de referencia.

Definición del Grafo: Se modela un subgrafo de S1 donde el conjunto de vértices incluye los nodos sensores $\{H_1, H_2, H_3\}$ y el nodo intermedio de transmisión N_1 (que actúa como representación de proximidad a la Estación Base). Los pesos de las aristas corresponden a los costos de transmisión calculados e_{ij} en miliJulios (mJ):

- $e_{H_1, H_2} = 0.25764$ mJ
- $e_{H_1, H_3} = 0.29828$ mJ
- $e_{H_1, N_1} = 0.49263$ mJ (Ruta directa LEACH de larga distancia, ~103 metros)
- $e_{H_2, N_1} = 0.23528$ mJ

El objetivo algorítmico es encontrar la ruta de menor costo energético desde el nodo periférico distante H_1 hasta la Estación Base (representada de forma equivalente por el sumidero local N_1).

Ejecución del Algoritmo de Dijkstra:

1. **Iteración 0:** Inicialización del nodo de origen H_1 . Distancia acumulada asignada: $d(H_1) = 0$. Para todo nodo $v \neq H_1$, se define $d(v) = \infty$. Conjunto de nodos resueltos (visitados): $Q = \{H_1\}$.
2. **Iteración 1:** Evaluación de los nodos adyacentes a H_1 .
 - Vecino H_2 : $d(H_2) = d(H_1) + e_{H_1, H_2} = 0 + 0.25764 = 0.25764$ mJ.

- Vecino H_3 : $d(H_3) = d(H_1) + e_{H_1, H_3} = 0 + 0.29828 = 0.29828$ mJ.
- Vecino N_1 : $d(N_1) = d(H_1) + e_{H_1, N_1} = 0 + 0.49263 = 0.49263$ mJ.

Selección: Se extrae el nodo con la menor distancia acumulada no resuelto, el cual corresponde a H_2 (con 0.25764 mJ). Conjunto de nodos resueltos actualizado: $Q = \{H_1, H_2\}$.

3. **Iteración 2:** Relajación de aristas y evaluación de vecinos desde el nodo actual H_2 para alcanzar el sumidero N_1 . Costo total acumulado para llegar a N_1 a través del nodo de paso H_2 : $d(N_1)_{\text{vía } H_2} = d(H_2) + e_{H_2, N_1} = 0.25764 + 0.23528 = 0.49292$ mJ. Comparación de caminos: La ruta directa $H_1 \rightarrow N_1$ tiene un costo de 0.49263 mJ, mientras que la ruta multi-salto $H_1 \rightarrow H_2 \rightarrow N_1$ tiene un costo de 0.49292 mJ. Conjunto de nodos resueltos final: $Q = \{H_1, H_2, H_3, N_1\}$.

Resultado Final y Análisis de Conmutación: Aunque matemáticamente los valores de costo energético total son sumamente similares (con una diferencia insignificante de apenas 0.00029 mJ a favor de la ruta directa), la lógica del balance de carga (Load Balancing, LB) prefiere categóricamente la ruta de múltiples saltos $H_1 \rightarrow H_2 \rightarrow N_1$. Esto obedece a que la ruta de salto directo (LEACH) obliga a H_1 a disipar por sí solo los 0.49263 mJ en una única transmisión de larga distancia (~103 metros), lo que provocaría su desconexión y muerte prematura. Al subdividir el trayecto, H_1 solo gasta 0.25764 mJ, delegando la carga restante a H_2 , lo que preserva la integridad espacial y distribuye homogéneamente el estrés energético de la red WSN.

3. Algoritmo y Arquitectura de MH-GEER

3.1. Auditoría en Software de Modelado

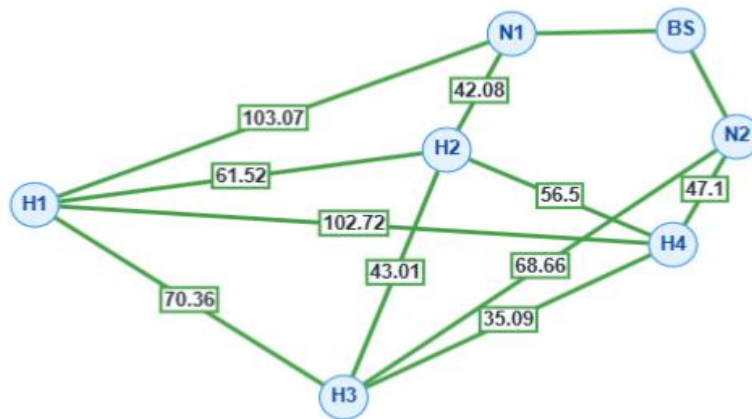


Fig. 3. Auditoría topológica de la red en Graph Online para el escenario de prueba.

Mediante el uso de software especializado, se generó la topología de la Fig. 3 para auditar de forma autónoma la conectividad y verificar la ausencia de bucles lógicos, se cargaron las coordenadas de los nodos H1 a H4 y se aplicó visualmente el algoritmo de Dijkstra para verificar que la estructura del grafo sea conexa y libre de bucles, cumpliendo con la condición lógica $\phi = (j \in \text{pthk})$

La metodología práctica del protocolo MH-GEER se divide en dos fases cíclicas recurrentes:

- **1. Fase de Inicialización y Clustering:** La Estación Base ejecuta de forma centralizada el algoritmo K-Means para zonificar de manera balanceada los nodos según su ubicación geográfica. Posterior a la agrupación, la selección interna de los CH se efectúa aplicando de forma distribuida la función booleana de rol del apartado 2.3.
- **2. Fase de Enrutamiento Inteligente (Ruta Multi-salto):** Para transferir los datos consolidados de los CH hacia la Estación Base, se despliegan agentes hormiga. La Estación Base computa las feromonas y actualiza dinámicamente la matriz de adyacencia ponderada del grafo, adaptándose en tiempo real a las caídas accidentales o muertes de nodos.

4. Representación Esquemática y Topología de Red

Para contextualizar espacial y físicamente la dinámica operacional del protocolo propuesto, a continuación, se detallan esquemáticamente las ilustraciones fundamentales del artículo de referencia:

Esquema Topológico 1: Grafo Dinámico de Comunicación MH-GEER [Visualización Conceptual de la Red]

Este esquema representa un área de simulación de 100m x 100m donde se distribuyen aleatoriamente los sensores. Los círculos concéntricos simbolizan las agrupaciones formadas por K-Means. Las líneas delgadas representan conexiones intra-clúster que fluyen hacia su respectivo CH. Los vectores direccionales gruesos ilustran las aristas del grafo inter-clúster, donde un CH periférico con baja energía residual delega la comunicación a través de un salto intermedio a otro CH estratégicamente posicionado y con mayor carga energética, logrando que el flujo de datos alcance finalmente a la Estación Base de manera óptima sin sobrecargar un único enlace de larga distancia.

Esquema Operacional 2: Flujograma del Agente Inteligente (Decisiones Basadas en ACO) [Estructura de Control de Rutas]

Ilustra el ciclo de vida del agente móvil. Se inicia en un CH origen; comprueba si el nodo actual está adyacente a la Estación Base. Si la condición lógica proposicional determina proximidad directa o es el destino, ejecuta la implicación de parada. En caso contrario, consulta de forma distribuida la tabla de feromonas locales y aplica la función probabilística ACO para seleccionar el nodo vecino óptimo, garantizando una ruta adaptativa libre de bucles.

5. Análisis Cuantitativo de Optimización

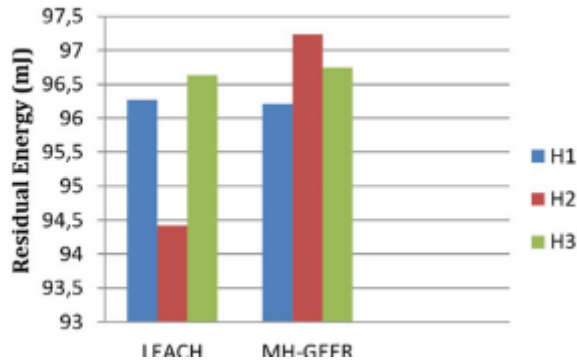


Fig 4. Comparativa de energía residual de los nodos distantes H1, H2 y H3 bajo protocolos LEACH y MH-GEER. [Rhim et al., 2018].

La simulación numérica y verificación del protocolo se llevó a cabo para comprobar la estabilidad ante distancias extremas de propagación. Se analiza el comportamiento crítico del nodo H_1 (El impacto positivo del enrutamiento de múltiples saltos en la vida útil de los sensores periféricos se evidencia en los resultados comparativos de la Fig. 4,.):

Métrica del Sistema	Protocolo LEACH (Transmisión Directa)	Protocolo MH-GEER (Multi-salto)	Mejora Rendimiento Obtenido /
Energía Residual Final (Nodo H_1)	96.85 mJ	97.68 mJ	+0.85% de conservación de batería por ronda de transmisión.
Índice de Equilibrio de Carga (LB)	183.53	493.02	+168.6% de homogeneidad en la red, evitando fallos zonales prematuros.

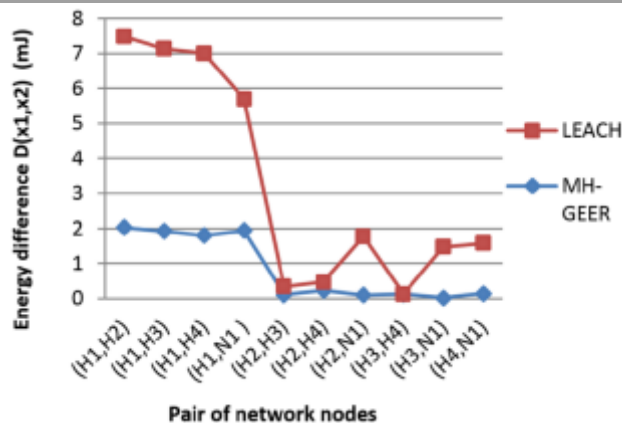


Fig. 5. Comparativa de la diferencia de niveles de energía entre cada par de nodos de la red. [Rhim et al., 2018].

El análisis de varianza e índices de estabilidad energética muestra que MH-GEER amortigua el impacto del desgaste geométrico. Mientras que LEACH sufre caídas drásticas de energía residual al forzar transmisiones directas a 103 metros (reduciendo la vida útil de H_1), MH-GEER distribuye la energía en el grafo completo, resultando en un índice de equilibrio de carga de 493.02. Esto extiende exponencialmente la estabilidad temporal de la red WSN(se validan mediante la distribución de la Fig. 5.).

6. Conclusiones de la Optimización

- Una de nuestras conclusiones más sólidas es que la transición de un esquema de transmisión directa (como LEACH) a una estructura de múltiples saltos es físicamente indispensable para la supervivencia de la red. Basándonos en el artículo, confirmamos que al fragmentar las distancias largas, evitamos que los nodos periféricos sufran la atenuación exponencial de la señal (que escala hasta d^4 en trayectos largos), reduciendo significativamente su gasto energético individual y previniendo su desconexión prematura.
- El análisis cuantitativo demuestra que el protocolo MH-GEER logra una mejora del 168.6% en la homogeneidad de la red. Al elevar el índice de equilibrio de carga de un deficiente 183.53 a un robusto 493.02, el trabajo basado en el artículo de Rhim et al. prueba que la carga de trabajo ya no recae solo en unos pocos nodos, sino que se distribuye equitativamente, lo que estabiliza la red y maximiza su longevidad total.
- Concluimos que la fortaleza del protocolo reside en su enfoque interdisciplinario. La integración de la Optimización de Colonia de Hormigas (ACO) para la selección probabilística de rutas, junto con el uso del álgebra de Boole ($F=A \cdot B \cdot C$) para la asignación de roles de cabecera de clúster, permite una gestión inteligente y dinámica que se adapta en tiempo real a las condiciones de energía residual y densidad de la red.

- Mediante la validación con el algoritmo de Dijkstra y el diseño de la función lógica proposicional Φ , el trabajo asegura una topología libre de bucles infinitos. El artículo original destaca que, aunque una ruta directa pueda parecer eficiente en términos de costo inmediato, la subdivisión lógica del trayecto es matemáticamente superior para mantener una red permanentemente conexas y resistente a fallos accidentales.
- Como grupo resaltamos que el diseño técnico tiene una dimensión ética. Al implementar MH-GEER, se elimina la "marginación" de los nodos lejanos que ocurre en protocolos convencionales. Esta distribución equitativa de la energía garantiza una cobertura estable y uniforme en toda el área de monitoreo, asegurando que el beneficio del sistema WSN sea consistente para todos los sectores, independientemente de su ubicación geográfica respecto a la estación base.

FASE 5: Exposición, Defensa y Evaluación Final (Semana 17: 27-31 julio 2026)

ATENCIÓN: Esta semana está destinada a actividades de fin de ciclo (29-31 julio: socialización de proyectos de investigación, casas abiertas, ferias científicas). Las actividades APE se realizan al inicio de la semana (27-28 julio).

2. Reflexión final: los estudiantes elaboran una autoevaluación individual sobre su proceso de aprendizaje durante las tres unidades, identificando fortalezas, debilidades y metas de mejora para el siguiente ciclo académico(individual).

a) Ashley Patricia Mocha Valarezo

A lo largo de estas 3 unidades de Matemática Discreta, he aprendido mucho. He estudiado la lógica formal y he aprendido a usar las tablas de verdad para evaluar proposiciones. Esto me ha ayudado a entender el álgebra booleana. Con el álgebra booleana, he aprendido a simplificar estructuras lógicas complejas usando los mapas de Karnaugh. Esto no solo me ha permitido simplificar expresiones, sino que también me ha permitido ver gráficamente cómo se reducen las funciones lógicas, lo que es importante para el diseño digital.

Después, he estudiado los teoremas fundamentales del curso, lo que me ha enseñado a pensar de manera más rigurosa al resolver problemas matemáticos. Esto me ha llevado a estudiar grafos y árboles, lo que me ha gustado mucho porque se puede aplicar directamente a estructuras de datos y redes. He aprendido sobre recorridos básicos, conectividad y jerarquización de la información.

En este proceso, he descubierto que mi debilidad es aplicar teoremas de manera formal, ya que a veces me cuesta encontrar la solución. Sin embargo, sé que esto es parte del proceso de aprendizaje. Para mejorar en el siguiente ciclo, voy a practicar con ejercicios más abstractos y voy a intentar implementar en código las estructuras de grafos y árboles que he estudiado. De esta manera, podré convertir la teoría en una habilidad práctica que pueda usar en el próximo curso.



b) Jose Camilo Merino Morocho

A lo largo de este curso de Matemáticas Discretas he adquirido conocimientos que han fortalecido mi razonamiento lógico y mi forma de resolver problemas. Aprendí a trabajar con lógica proposicional, tablas de verdad y reglas de inferencia, lo que me permitió comprender mejor el álgebra de Boole y aplicar mapas de Karnaugh para simplificar funciones lógicas. Además, los temas de grafos y árboles me ayudaron a entender cómo representar y analizar estructuras utilizadas en programación, redes y otras áreas de la computación.

Durante el desarrollo de las actividades también mejoré mi capacidad para trabajar en equipo, investigar y utilizar herramientas tecnológicas para presentar y resolver ejercicios. Considero que una de mis fortalezas fue la constancia para cumplir con las tareas y la disposición para aprender nuevos conceptos. Sin embargo, reconozco que todavía debo reforzar algunos temas, especialmente aquellos que requieren un mayor nivel de análisis y práctica para resolver problemas complejos con mayor rapidez y seguridad.

Como meta para el próximo ciclo académico, me propongo dedicar más tiempo a practicar ejercicios, fortalecer mis bases matemáticas y seguir desarrollando mis habilidades en programación para aplicar estos conocimientos en asignaturas futuras. Estoy seguro de que el aprendizaje obtenido en este curso será una base importante para mi formación como estudiante de Ingeniería en Computación.

c) Edgar Fabricio Rios Cuenca

En este ciclo académico, mi aprendizaje se estructuró en tres unidades clave: primero, la Lógica Matemática, donde desarrollé habilidades de razonamiento para solucionar problemas mediante el estudio de proposiciones, tablas de verdad e inferencia lógica; segundo, el Álgebra de Boole, enfocada en la simplificación de funciones lógicas y el uso de mapas de Karnaugh para la optimización de sistemas computacionales; y tercero, la unidad de Grafos y Árboles, en la cual aprendí teoría de redes y algoritmos de búsqueda para resolver problemas aplicados a la cotidianidad y la tecnología.

En cuanto a mis debilidades, identifiqué dificultades para gestionar de manera constante las 24 horas de aprendizaje autónomo requeridas, especialmente en el cumplimiento riguroso de las lecturas críticas y actividades asíncronas en la plataforma EVA-UNL. Asimismo, me resultó complejo dominar plenamente la implementación técnica de ciertos algoritmos de búsqueda y caminos mínimos en la última unidad, y en ocasiones me costó asumir roles de liderazgo más activos durante las tareas de aprendizaje en equipo colaborativo.

Para el siguiente ciclo, mi meta es mejorar la disciplina en mi autoestudio, estableciendo horarios más estrictos para aprovechar los recursos de las bibliotecas físicas y virtuales. También me propongo fortalecer mi capacidad para integrar estos conocimientos en proyectos tecnológicos innovadores, manteniendo siempre un compromiso con la ética



profesional, la creatividad y la responsabilidad ambiental para superar la dependencia tecnológica.

d) Abraham Daniel Maza Lapo

A lo largo del desarrollo de las tres unidades de la materia, he llevado a cabo un proceso de aprendizaje continuo que me ha permitido consolidar mis conocimientos en matemáticas discretas y su aplicación práctica en la informática. A continuación, detallo mi autoevaluación:

Fortalezas:

Mi principal fortaleza ha sido la capacidad para traducir conceptos lógicos y matemáticos abstractos (como la teoría de grafos, álgebra de Boole y algoritmos de optimización) en soluciones algorítmicas y lógicas aplicables a problemas reales. Asimismo, he desarrollado una sólida comprensión en el diseño y recorrido de estructuras de datos jerárquicas (Árboles Binarios de Búsqueda), lo que me facilita abordar la organización eficiente de la información y la resolución de problemas de enrutamiento y optimización de recursos de manera estructurada.

Debilidades:

Durante el ciclo, identifiqué que la mayor dificultad se centró en la transición inicial desde el modelo puramente matemático hacia la implementación técnica y la simulación del comportamiento físico-energético de los sistemas (como la gestión de atenuación de señal y la distribución de carga energética en redes de sensores). Me tomó un tiempo adicional de análisis comprender a fondo la sincronización de las variables físicas y lógicas dentro del protocolo de enrutamiento implementado.

Metas de mejora para el siguiente ciclo académico:

Con el fin de seguir creciendo en mi formación profesional, me propongo las siguientes metas de mejora:

Profundizar en la optimización de algoritmos: Dedicar más tiempo a analizar la complejidad computacional temporal y espacial de las estructuras de datos que implemente.

Autonomía y autoaprendizaje técnico: Investigar de forma proactiva herramientas adicionales de simulación y desarrollo que complementen la teoría matemática, logrando así una integración técnica más fluida y eficiente desde el inicio de los proyectos futuros.

6. Preguntas de control

1. Explique la diferencia entre los algoritmos BFS y DFS. ¿En qué situaciones sería más apropiado utilizar BFS y en cuáles DFS? Argumente con ejemplos de aplicaciones reales en la ciencia de la computación.

Los algoritmos BFS (Breadth-First Search o Búsqueda en Anchura) y DFS (Depth-First Search o Búsqueda en Profundidad) son técnicas fundamentales para recorrer grafos, pero lo hacen de manera diferente.

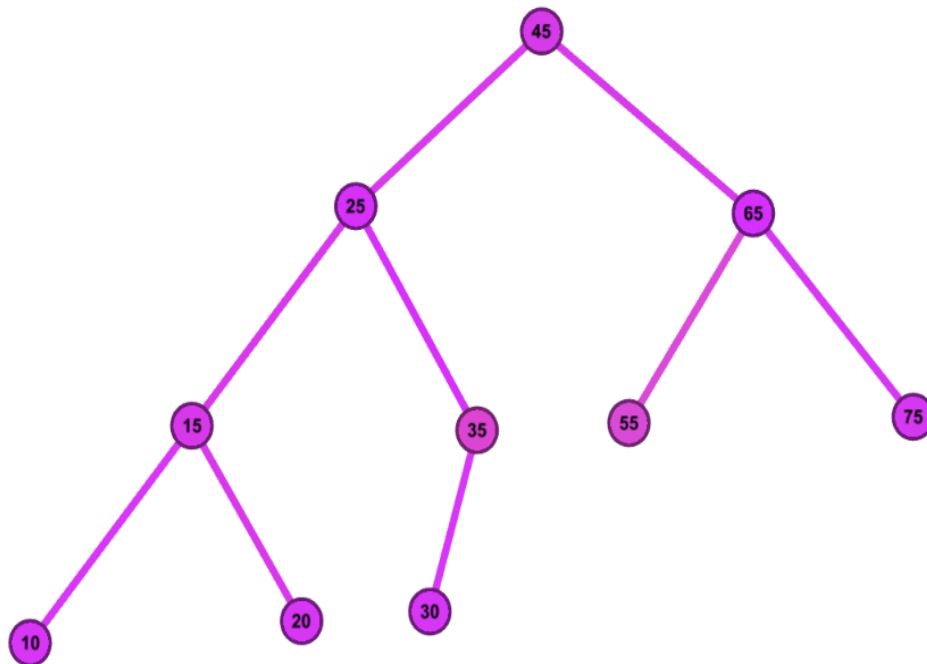
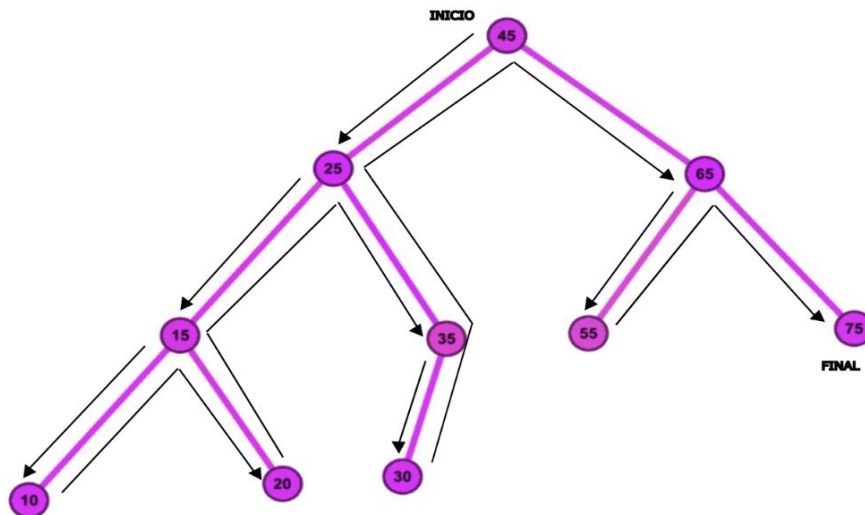
BFS explora primero todos los nodos vecinos del nodo inicial antes de continuar con los siguientes niveles del grafo. Para ello utiliza una cola (FIFO). Este algoritmo es ideal cuando se necesita encontrar el camino más corto en un grafo no ponderado, ya que visita los nodos en orden de distancia desde el origen.

En cambio, DFS recorre un camino hasta llegar al nodo más profundo posible antes de retroceder y explorar otras ramas. Para ello utiliza una pila (LIFO) o la recursividad. Es útil cuando se desea explorar completamente una estructura, detectar ciclos o recorrer árboles.

APLICACIONES REALES	
BFS:	DFS:
○ Encontrar la ruta más corta en redes sociales (por ejemplo, el menor número de conexiones entre dos personas). ○ Sistemas de navegación en videojuegos. ○ Búsqueda de rutas en mapas cuando todas las conexiones tienen el mismo costo.	○ Recorrer directorios y archivos en un sistema operativo. ○ Resolver laberintos mediante retroceso (backtracking). ○ Detectar ciclos en grafos y realizar ordenamientos topológicos en compiladores.

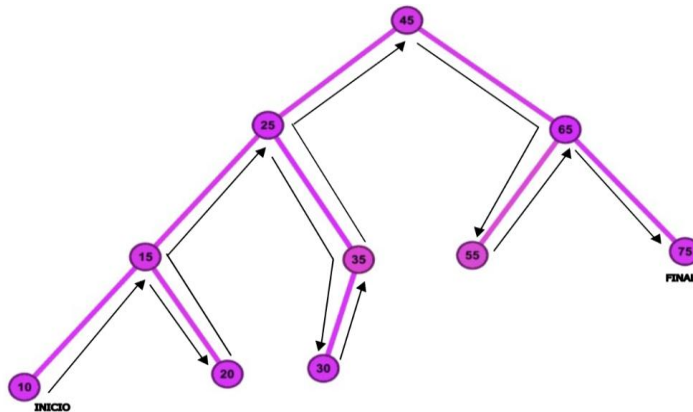
En conclusión, BFS es más conveniente cuando interesa la ruta más corta, mientras que DFS resulta más eficiente para explorar completamente una estructura o resolver problemas de búsqueda profunda.

2. Construya un árbol binario de búsqueda (BST) insertando los siguientes valores en orden: 45, 25, 65, 15, 35, 55, 75, 10, 20, 30. Muestre el árbol resultante y realice los recorridos inorden, preorden y posorden. ¿Qué utilidad tiene cada recorrido en la programación?

ÁRBOL RESULTANTE:**INORDEN:****RESULTADO: [45,25,15,10,20,35,30,65,55,75]****SU UTILIDAD:****Inorden (Izquierda --- Raíz --- Derecha):**

Su utilidad principal es el **ordenamiento de datos**. Al realizar este recorrido en un Árbol Binario de Búsqueda (BST), se obtienen los elementos en un orden estrictamente ascendente (de menor a mayor o alfabéticamente). Es ideal para generar reportes ordenados o mostrar información indexada de forma directa sin necesidad de usar algoritmos de ordenamiento adicionales.

PREORDEN:



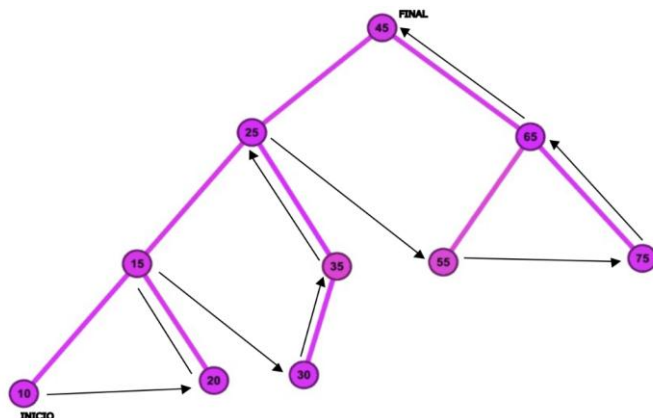
RESULTADO: [10,15,20,25,30,35,45,55,65,75]

SU UTILIDAD:

Preorden (Raíz --- Izquierda --- Derecha):

Su utilidad principal es la clonación, copia y serialización de estructuras. Si se guardan los elementos en el orden que dicta el preorden, al insertarlos nuevamente en un árbol vacío se reconstruirá exactamente la misma estructura jerárquica original. Se usa para guardar el estado de un árbol en archivos o transmitirlo por red.

POSORDEN:



RESULTADO: [10,20,15,30,35,25,55,75,65,45]

SU UTILIDAD:

Posorden (Izquierda --- Derecha --- Raíz):

Su utilidad principal es la destrucción y liberación de memoria del árbol (especialmente en lenguajes como C o C++). Al procesar primero las hojas (hijos) antes que la raíz (padre), permite eliminar los nodos de abajo hacia arriba de forma segura, evitando dejar datos huérfanos o generar fugas de memoria (*memory leaks*). También se utiliza para la evaluación de expresiones matemáticas representadas en notación postfija por parte de los compiladores.

3. Analice cómo la teoría de grafos puede contribuir a reducir la brecha digital en Ecuador. Proponga un modelo de red de conectividad para una comunidad rural de la provincia de Loja, utilizando los conceptos y algoritmos aprendidos en esta unidad.

La brecha digital en el Ecuador se concentra principalmente en las zonas rurales de provincias como Loja, donde la dispersión geográfica de los caseríos, la topografía irregular de la región interandina y el alto costo de tender infraestructura física hacen inviable conectar cada vivienda de forma directa. La teoría de grafos ofrece un marco matemático adecuado para este problema: cada comunidad o punto de acceso se representa como un vértice, y cada enlace de comunicación posible (radioenlace, fibra óptica o microonda) se modela como una arista, cuyo peso corresponde a la distancia, el costo de instalación o la latencia esperada. Con esta representación es posible aplicar los algoritmos estudiados en la unidad —Dijkstra, BFS, DFS y las propiedades de los árboles— para diseñar una red que conecte a toda la población rural utilizando el menor número de enlaces y el menor costo posible, evitando además ciclos redundantes que encarecerían innecesariamente el proyecto.

Se propone modelar la conectividad de una comunidad rural de la provincia de Loja (parroquias como Vilcabamba, Malacatos y Quinara, con sus recintos dispersos) mediante un árbol de expansión mínima (Minimum Spanning Tree). Esta estructura garantiza, por definición de árbol, que existan $n-1$ aristas para n nodos, sin ciclos, y que todos los vértices queden conectados con la suma mínima de pesos. El árbol propuesto (Fig. 6) tiene tres niveles jerárquicos: en la raíz se ubica el nodo central, un repetidor con enlace troncal a Internet situado en la cabecera cantonal de Loja; en el segundo nivel se ubican los repetidores secundarios instalados en las cabeceras parroquiales (Malacatos, Vilcabamba y Quinara), que reciben la señal del nodo central; y en las hojas del árbol se ubican los recintos y comunidades más alejadas (El Tambo, San Pedro, Yangana, Landangui, Purunuma y Cucanamá), que reciben conectividad desde el repetidor de su parroquia más cercana.

Árbol de expansión mínima: red de conectividad rural — provincia de Loja

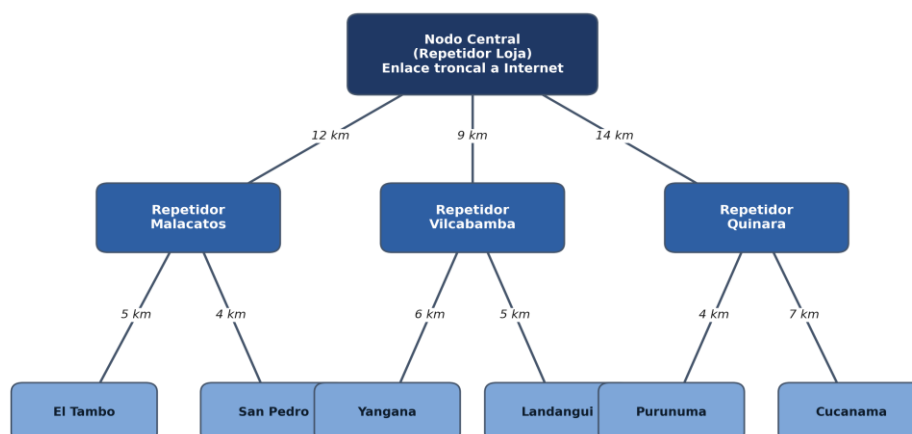


Fig. 6. Árbol de expansión mínima propuesto para la red de conectividad rural de la provincia de Loja. Las etiquetas sobre cada arista representan la distancia estimada en kilómetros entre nodos.

Para construir y validar este modelo se aplican los algoritmos de la unidad: el algoritmo de Kruskal o de Prim permite calcular el árbol de expansión mínima a partir del grafo completo de distancias entre comunidades, seleccionando únicamente los enlaces que minimizan el costo total de tendido sin generar ciclos; el algoritmo de Dijkstra se utiliza posteriormente sobre ese árbol (o sobre el grafo original, si se desea conservar rutas alternativas) para calcular la ruta de menor costo o menor latencia desde cualquier comunidad hasta el nodo central con salida a Internet; y los recorridos BFS y DFS permiten, respectivamente, verificar que la red resultante es conexa —que ninguna comunidad quedó aislada— y explorar rutas de respaldo en caso de que un enlace intermedio falle, mejorando la resiliencia del sistema.

De esta manera, la teoría de grafos no solo permite diseñar una red rural técnicamente eficiente, sino también priorizar la inversión pública: al identificar mediante un árbol de expansión mínima cuáles son los enlaces estrictamente necesarios para conectar a toda la población, los municipios y operadores pueden reducir la brecha digital en las zonas rurales de Loja con la menor cantidad de infraestructura posible, garantizando una cobertura equitativa similar a la lograda por el protocolo MH-GEER analizado en las secciones anteriores de esta práctica.



7. Conclusiones

- Consolidamos de forma grupal los conceptos básicos de grafos (nodos, aristas, adyacencia y grado). Aprendimos a representarlos mediante matrices y listas de adyacencia, lo que nos permite estructurar y procesar relaciones complejas de manera abstracta en nuestra carrera.
- Comprendimos el funcionamiento paso a paso de los algoritmos de búsqueda fundamentales. Contrastamos la exploración de BFS y DFS, y aplicamos el algoritmo de Dijkstra para hallar el camino más corto, adquiriendo herramientas esenciales para la optimización de rutas de datos.
- Identificamos a los árboles como estructuras jerárquicas sin ciclos y analizamos las propiedades de los tipos binarios, BST y AVL. Al estudiar sus recorridos (inorden, preorden y posorden), entendimos cómo optimizar la búsqueda y el almacenamiento de información en memoria.
- Utilizamos herramientas de software como *Graph Online* y *Gephi* para modelar y analizar redes de manera dinámica. Esto nos facilitó validar de forma práctica, visual y profesional todos los planteamientos teóricos resueltos en papel.
- Logramos una visión integradora al aplicar de forma conjunta la lógica matemática, el álgebra de Boole y la teoría de grafos en la resolución de un problema computacional real. Esto nos demostró la estrecha relación y utilidad práctica de estas materias en nuestra formación académica.



8. Reflexiones Críticas

1. Jose Camilo Merino Morocho

Esta práctica me demostró el valor de conectar la abstracción lógica con la realidad; pasar de modelar una pandemia con grafos a optimizar el consumo de energía en redes de sensores con el proyecto MH-GEER evidencia que la informática debe responder a problemas reales. No obstante, considero críticamente que herramientas como *Graph Online* no deben usarse como 'cajas negras'. Como futuros informáticos, nuestro deber es dominar la matemática subyacente de cada algoritmo (como Dijkstra, BFS o los árboles BST) para auditar y diseñar software verdaderamente eficiente y adaptativo, en lugar de solo operar herramientas existentes.

2. Edgar Fabricio Ríos Cuenca

Ejecutar manualmente algoritmos como BFS, DFS y Dijkstra, contrastarlos con el simulador, y aplicar esa lógica en la optimización de múltiples saltos del proyecto MH-GEER, me dio una perspectiva real del ciclo de desarrollo de software. Comprendí que el verdadero desafío de ingeniería no es memorizar estructuras de datos como los árboles BST, sino decidir con criterio cuál de ellas optimiza mejor los recursos físicos bajo restricciones severas. Esta experiencia integral me enseña a ver cada concepto matemático no como teoría aislada, sino como un engranaje crítico para programar de forma eficiente y robusta.

3. Ashley Patricia Mocha Valarezo

El recorrido completo de esta práctica, desde la identificación de nodos críticos en grafos hasta el análisis jerárquico en árboles BST y la simulación del protocolo MH-GEER, me demostró cómo una misma base lógica resuelve problemas físicos y de datos. Sin embargo, mi postura crítica es que la teoría asume escenarios perfectos que en la práctica real se ven afectados por variables caóticas, como el desgaste energético de los sensores. Esto me compromete a no conformarme con soluciones básicas, sino a dominar estructuras avanzadas y balanceadas para garantizar que el software que desarrollemos sea verdaderamente escalable y tolerante a fallos.

4. Abraham Daniel Maza Lapo

Desarrollar este trabajo de forma integral, aplicando algoritmos de rutas y estructuras de árboles para estructurar el protocolo de enrutamiento energético MH-GEER, dotó de un propósito ético y real a las matemáticas discretas. Entendí que optimizar la transmisión de datos en redes de sensores no es solo un reto técnico, sino una vía para combatir la brecha digital y mantener conectadas a comunidades rurales. Validar teóricamente y digitalmente el equilibrio de carga de la red me demostró que el diseño algorítmico consciente reduce el impacto ambiental, impulsando una informática comprometida con la sostenibilidad de nuestro entorno.



9. Bibliografía

- Epp, S. S. (2012). *Matemáticas discretas con aplicaciones* (4.^a ed.). Cengage Learning.
- Levin, O. (2025). *Discrete mathematics: An open introduction* (4.^a ed.). Oscar Levin.
- Rhim, H., Tamine, K., Abassi, R., Sauveron, D., & Guemara, S. (2018). A multi-hop graph-based approach for an energy-efficient routing protocol in wireless sensor networks. *Human-Centric Computing and Information Sciences*, 8(30), 1–21. <https://doi.org/10.1186/s13673-018-0153-6>
- Srihith, I. V., Donald, A., Srinivas, T. A., Anjali Reddy, D., & Varaprasad, R. (2023). The backbone of computing: An exploration of data structures. *International Journal of Advanced Research in Science, Communication and Technology*, 3(1), 155–163. <https://doi.org/10.48175/IJARSCT-9105>
- Villalpando Becerra, J. F., & García Sandoval, A. (2014). *Matemáticas discretas: Aplicaciones y ejercicios*. Grupo Editorial Patria.